

The Chess Algorithm: A Novel Metaheuristic Optimization Technique with Applications in Transportation Network Engineering

Seyed Saber Naserlavi

Seyedali Mirjalili

2026-07-04

We introduce the Chess Algorithm (CA), a population-based metaheuristic that borrows its search logic from the strategy of chess rather than from foraging or hunting behavior in nature. The population is not homogeneous: at every iteration agents are ranked and assigned the roles of King, Queens, Rooks, Bishops, Knights, and Pawns, and each role moves according to an operator abstracted from the geometry of the corresponding piece. Strategic ideas from the game supply the control machinery, and a lightweight state machine reads the divergence, initiative, and local-search success of the population each iteration to decide which of them apply: development, sacrifice, pinning, castling, en passant, and threefold repetition throughout; Novotny interference, the knight's fork, the royal council, and a blockade response combining a King's march with a Tal-style speculative sacrifice in the tactically richer phases of the search. On six standard 30-dimensional benchmarks against GA, PSO, SA, and GWO under matched evaluation budgets, CA is significantly better than PSO and SA on every function and than GA on five of six (tied on the sixth), while GWO keeps its known edge on this classical suite. On the full CEC-2017 benchmark suite (24 functions validated after a two-stage data-integrity audit that identified and excluded five defective third-party implementations, $D = 30$, 30 independent runs), CA achieves the best mean Friedman rank among six algorithms including GWO, PSO, GA, and WOA, with zero losses to GWO or WOA and only three of twenty-four to PSO; a specific, honestly reported trade-off against GA on long-range multimodal landscapes is read as a clean illustration of the No-Free-Lunch theorem rather than argued away. On seven classic constrained engineering design problems under the same roster, CA again ranks first, losing exactly one of thirty-five pairwise comparisons. Practical value is assessed on transportation problems: on the coordinated signal timing of an eight-intersection arterial with a Webster/HCM delay model, CA is significantly better than GA and GWO; on a continuous berth allocation instance with a known global optimum, an extended comparison against six algorithms from the independent mealpy library assesses CA's placement against third-party implementations rather than our own. Throughout, an ablation configuration with the adaptive control layer disabled (CA-static) is carried alongside CA to isolate what that layer contributes. The results support a measured claim: CA is a sound, competitive, adaptively self-regulating, and fully reproducible addition to the toolbox of global and transportation-network optimization.

¹ Seyed Saber Naserlavi (corresponding author, saber_naserlavi@uk.ac.ir) — Department

of Civil Engineering, Faculty of Engineering, Shahid Bahonar University of Kerman, Kerman, Iran.

² Seyedali Mirjalili — Centre for Artificial Intelligence Research and Optimization, Torrens University Australia, Brisbane, Australia.

An HTML version of this article, together with all code and data, is available at https://sabernaserlavi-60.github.io/2026_Chess-Algorithm/.

Author note. Dr. Seyedali Mirjalili is listed as an invited co-author; his participation is pending confirmation. The invitation reflects how much this study owes to his work on swarm and nature-inspired optimization, and his name will be retained only with his explicit consent.

Keywords: metaheuristics; chess algorithm; adaptive optimization; global optimization; CEC-2017 benchmarks; constrained engineering design; traffic signal timing; berth allocation; transportation networks.

1 Introduction

1.1 Motivation

A large share of the decisions a transportation engineer has to make can be written, after enough simplification, as a global optimization problem:

$$\min_{\mathbf{x} \in \Omega} f(\mathbf{x}), \quad \Omega = \{\mathbf{x} \in \mathbb{R}^D : l_j \leq x_j \leq u_j, j = 1, \dots, D\}. \quad (1)$$

The trouble is what f looks like in practice. Signal timing, network design, transit scheduling, detector placement — in each of these the objective is non-convex, often non-differentiable, riddled with local optima, and sometimes expensive to evaluate (Ceylan & Bell, 2004; Osorio & Bierlaire, 2013). Exact methods rarely scale to instances of realistic size. This is why metaheuristics have kept their popularity for three decades: they trade the guarantee of optimality for robustness, generality, and a computational budget one can actually afford.

1.2 A brief map of the metaheuristic landscape

The field grew from two roots. Evolutionary computation gave us the Genetic Algorithm (Goldberg, 1989; Holland, 1975) and Evolution Strategies (Beyer & Schwefel, 2002). Trajectory methods gave us Simulated Annealing (Kirkpatrick et al., 1983), whose central idea — accept a worse solution now and then, so you can escape a local minimum — turned out to be one of the most durable in the field. Swarm intelligence arrived in the 1990s with Particle Swarm Optimization (Kennedy & Eberhart, 1995), Ant Colony Optimization (Dorigo et al., 1996), Differential Evolution (Storn & Price, 1997), and later the Artificial Bee Colony (Karaboga & Basturk, 2007). More recently, Mirjalili and colleagues built a productive line of algorithms by abstracting the social and hunting behavior of animals into compact operators: the Grey Wolf Optimizer (Mirjalili et al., 2014), the Ant Lion Optimizer (Mirjalili, 2015b), the Dragonfly Algorithm (Mirjalili, 2016a), the Whale Optimization Algorithm (Mirjalili & Lewis, 2016), the Sine Cosine Algorithm (Mirjalili, 2016b), and the Salp Swarm Algorithm (Mirjalili et al., 2017), among others.

Reading this literature, two things stand out. First, almost every swarm algorithm treats its agents identically: one update equation, applied to everyone, with only position and fitness distinguishing one agent from the next. Yet division of labor among *unequal* agents is one of the oldest tricks in both nature and human strategy. Second, the quality of a metaheuristic usually stands or falls on how it schedules the transition from exploration to exploitation (Mirjalili et al., 2014), and mechanisms that adapt this transition to what the search is actually observing — rather than to elapsed time alone — remain surprisingly rare.

We should also be upfront about why proposing yet another metaheuristic is defensible at all. The No-Free-Lunch theorem (Wolpert & Macready, 1997) rules out a universally dominant optimizer, so the honest goal is not to win everywhere; it is to contribute a genuinely different inductive bias and to show the problem class where that bias earns its keep.

1.3 Why chess?

Chess offers exactly the two ingredients we just found missing. Its pieces are radically unequal: the queen sweeps the whole board, rooks travel along files and ranks, bishops along diagonals, knights jump over anything in the way, and pawns crawl forward one square at a time while quietly defining the structure of the position. Mapped onto a search population, this suggests sub-groups performing qualitatively different moves around one common reference point — the King, which in our setting is the best solution found so far.

The game also comes with a mature strategic vocabulary, refined over centuries of analysis. Development says: mobilize your forces early, consolidate later. A sacrifice gives up material now for a positional advantage later. Pinning immobilizes a piece against something more valuable behind it. Castling is a safeguarded structural rearrangement; en passant, an opportunistic capture available only under narrow conditions; the threefold-repetition rule declares a draw when the position keeps repeating. Each of these has a natural algorithmic reading — exploration schedules, acceptance of deteriorating moves, variable fixation, elite restructuring, adaptive local search, and diversity preservation, respectively.

The Chess Algorithm (CA) proposed here formalizes that mapping. To avoid a misunderstanding that came up more than once when we described the idea to colleagues: CA does not play chess and searches no game tree. It borrows the strategic grammar of the game, nothing more, and its per-iteration cost is comparable to that of GA or PSO.

1.4 Contributions and scope

The paper contributes, in order: a complete mathematical specification of CA — initialization, rank-based role assignment, movement operators, strategic mechanisms, and the adaptive control layer that governs when each applies, including a state machine that reads population divergence, initiative, and local-search success each iteration and switches between opening, middlegame, closed-position, and endgame tactical repertoires, with several operators new to this paper (Novotny interference, the knight’s fork, the royal council, a King’s-march blockade response, and a Tal-style speculative-sacrifice reheat) (Section 2); a controlled benchmark study on six standard test functions in 30 dimensions against GA, PSO, SA, and GWO under matched budgets, with nonparametric testing following Derrac et al. (2011) (Section 3); a full evaluation on the CEC-2017 benchmark suite against GWO, PSO, GA, and WOA, including a data-integrity audit that identified five defective functions in the third-party `opfunu` reference implementation (Section 4); an evaluation on seven classic constrained engineering design problems against the same roster (Section 5); a transportation case study on coordinated signal timing of an

eight-intersection arterial with a Webster/HCM delay model (Roess et al., 2019; Webster, 1958) (Section 6); an extended comparison against six third-party implementations from the `mealpy` library on two transportation problems, one of which has a known global optimum (Section 7); and a fully reproducible open-source implementation, with an ablation configuration (CA-static, the adaptive layer disabled) carried throughout to isolate its contribution.

A remark on tone before we start. We report results with deliberate restraint and no claim of universal superiority; GWO, in particular, remains stronger on several classical benchmarks, and we say so plainly where it happens. The claim we do defend is narrower: CA is well-founded, novel, and competitive, and on the transportation problems that motivated it in the first place it performs on par with — and against some widely used algorithms, better than — the available alternatives.

2 The Chess Algorithm

2.1 Conceptual mapping

CA maintains a population of N candidate solutions $\mathbf{X}_i \in \Omega \subset \mathbb{R}^D$, treated as pieces playing “toward” the global optimum. Table 1 summarizes how the vocabulary of the game maps onto algorithmic mechanisms.

Table 1: Mapping between chess concepts and CA mechanisms.

Chess concept	Algorithmic role
King	Incumbent best solution (never lost)
Queen, Rook, Bishop, Knight, Pawn	Heterogeneous movement operators
Development (opening principle)	Time-decaying step-scale $a(t)$
Sacrifice	Probabilistic acceptance of worse moves (minor pieces only)
Pinning	Progressive freezing of coordinates to the King’s values
Castling	Safeguarded coordinate-block exchange King $\leftrightarrow$ Rook
En passant	Opportunistic, self-adaptive local capture around the King
Pawn promotion	Rank-based role reassignment each iteration
Threefold repetition	Re-deployment of agents that collapse onto the King
Opening / middlegame / closed position / endgame	Position-dependent tactical phase, read each iteration (Section 2.11)
Nimzowitsch prophylaxis	Restrained, line-reopening response to premature convergence
Fork, interference, discovered attack, royal council	Additional movement and probe operators, active in specific phases
Windmill	Repeated extension of a successful King displacement (endgame)
Zugzwang / blockade	Stagnation-triggered waiting move, King’s march, and speculative sacrifice
Checkmate	Termination criterion

One point deserves emphasis: CA borrows the strategic grammar of chess as an inductive bias, not the game itself. An iteration costs $\mathcal{O}(ND + N \log N)$ arithmetic operations (the movement updates plus the sort that drives role assignment) and $N + 3$ function evaluations — N piece moves plus three en-passant probes — with one extra castling probe every c iterations. That is the same order as GA, PSO, or GWO. The exact evaluation accounting used in the experiments is spelled out in Section 3.

2.2 Initialization

The board is set up by scattering the N pieces uniformly over the feasible box:

$$\mathbf{X}_i^{(0)} = \mathbf{l} + (\mathbf{u} - \mathbf{l}) \circ \mathbf{r}_i, \quad \mathbf{r}_i \sim \mathcal{U}(0, 1)^D, \quad i = 1, \dots, N, \quad (2)$$

where $\mathbf{l}$ and $\mathbf{u}$ are the bound vectors and $\circ$ is the elementwise (Hadamard) product. All positions are evaluated once and the population is sorted by fitness before the first move is played.

2.3 Role assignment

Let π be the permutation of $\{1, \dots, N\}$ that sorts the population,

$$f(\mathbf{X}_{\pi(1)}) \leq f(\mathbf{X}_{\pi(2)}) \leq \dots \leq f(\mathbf{X}_{\pi(N)}). \quad (3)$$

The best agent $\mathbf{K} = \mathbf{X}_{\pi(1)}$ is the King. The following ranks are partitioned, in order, into Queens $\mathcal{Q}$, Rooks $\mathcal{R}$, Bishops $\mathcal{B}$, and Knights $\mathcal{N}$, with

$$|\mathcal{Q}| = \max(1, \lfloor \rho_q N \rfloor), \quad |\mathcal{R}| = \max(1, \lfloor \rho_r N \rfloor), \quad |\mathcal{B}| = \max(1, \lfloor \rho_b N \rfloor), \quad |\mathcal{N}| = \max(1, \lfloor \rho_n N \rfloor),$$

where $\lfloor \cdot \rfloor$ denotes rounding and $(\rho_q, \rho_r, \rho_b, \rho_n) = (0.10, 0.15, 0.15, 0.20)$. Whatever remains becomes the Pawns $\mathcal{P}$. Because the assignment is redone after every iteration, a pawn that stumbles into a good region is simply a strong piece the next time roles are dealt. Promotion, in other words, falls out of re-ranking for free; no extra machinery is needed.

2.4 Development schedule

Opening theory tells a player to develop pieces quickly; endgames reward patience and precision. CA encodes this with a linearly decaying step-scale coefficient,

$$a(t) = 2 \left(1 - \frac{t}{T} \right), \quad (4)$$

with t the current iteration and T the total budget — the convention popularized by Mirjalili et al. (2014). Every piece move scales with $a(t)$, so the swarm plays big developing moves early and fine maneuvers late.

2.5 Piece movement operators

Write $\mathbf{L} = \mathbf{u} - \mathbf{l}$ for the box width and let $\mathbf{r}, \mathbf{r}' \sim \mathcal{U}(0, 1)^D$ be independent random vectors. The five mobile roles then move as follows.

Queen (omnidirectional sweep). The queen circles the King at a radius tied to her current distance from him:

$$\mathbf{X}'_i = \mathbf{K} + a(t) (2\mathbf{r} - \mathbf{1}) \circ \max(|\mathbf{K} - \mathbf{X}_i|, \sigma), \quad (5)$$

where σ is the King's adaptive capture radius from Section 2.6.3. The max keeps the sweep from collapsing entirely, so queens never stall.

Rook (single-axis move). One random axis j is chosen and only that coordinate moves:

$$X'_{i,j} = K_j + a(t) (2r - 1) \max(|K_j - X_{i,j}|, 0.01 L_j a(t)). \quad (6)$$

Bishop (diagonal move). Two distinct axes j and k receive steps of equal magnitude and random relative sign, which is as close to a diagonal as a box-constrained space allows:

$$\Delta = a(t) (2r - 1) \max\left(\frac{1}{2}(|K_j - X_{i,j}| + |K_k - X_{i,k}|), 0.01 L a(t)\right), \quad X'_{i,j} = K_j + \Delta, \quad X'_{i,k} = K_k \pm \Delta. \quad (7)$$

Knight (L-shaped jump). The knight is the one piece that jumps over whatever stands in its way. Relative to a randomly chosen better-ranked peer $\mathbf{P}$, it drifts toward the peer and then adds an asymmetric 2:1 jump on two random axes:

$$\mathbf{X}'_i = \mathbf{X}_i + \mathbf{r} \circ (\mathbf{P} - \mathbf{X}_i), \quad X'_{i,j} += 2 a(t) \beta (2r - 1), \quad X'_{i,k} += 1 a(t) \beta (2r' - 1), \quad (8)$$

with $\beta = \overline{|\mathbf{P} - \mathbf{X}_i|}$ the mean coordinate gap. With small probability $0.1 a(t)/2$ the knight abandons this move and leaps: two coordinates are resampled uniformly in Ω . This leap is the algorithm's main long-range restart device. The implementation also ships a heavy-tailed variant in which those two coordinates instead receive a Lévy-flight step $0.05 L s$ generated by Mantegna's algorithm (Mantegna, 1994), the mechanism Cuckoo Search made popular (Yang & Deb, 2009):

$$s = \frac{u}{|v|^{1/\beta_\ell}}, \quad u \sim \mathcal{N}(0, \sigma_u^2), \quad v \sim \mathcal{N}(0, 1), \quad \sigma_u = \left[\frac{\Gamma(1 + \beta_\ell) \sin(\pi \beta_\ell / 2)}{\Gamma(\frac{1 + \beta_\ell}{2}) \beta_\ell 2^{(\beta_\ell - 1)/2}} \right]^{1/\beta_\ell}, \quad (9)$$

with tail index $\beta_\ell = 1.5$. Every result in this paper uses the plain uniform leap; the Lévy option is documented for future study rather than used here.

Pawn (steady advance). Pawns take small steps toward the King and toward a random better-ranked piece $\mathbf{B}$:

$$\mathbf{X}'_i = \mathbf{X}_i + 0.3 \mathbf{r} \circ (\mathbf{K} - \mathbf{X}_i) + 0.3 \mathbf{r}' \circ (\mathbf{B} - \mathbf{X}_i). \quad (10)$$

2.6 Strategic mechanisms

2.6.1 Pinning

With probability $p_{\text{pin}}(t) = 0.5 t/T$, a random subset of at most 30% of an agent’s coordinates is pinned to the King’s values and excluded from perturbation. Early on, pinning is rare and the pieces develop freely. Late in the run it concentrates the search on a shrinking set of free dimensions around the incumbent, which is exactly the kind of scheduled exploitation an endgame calls for.

2.6.2 Sacrifice

A worsening move $\Delta_i = f(\mathbf{X}'_i) - f(\mathbf{X}_i) > 0$ made by a minor piece — a Knight or a Pawn — is still accepted with probability

$$p_{\text{acc}} = \exp\left(-\frac{\Delta_i}{\theta(t)}\right), \quad \theta(t) = \theta_0(f_{\text{max}} - f_{\text{min}}) e^{-8t/T}, \quad (11)$$

with $\theta_0 = 0.1$. Readers will recognize the Metropolis rule of Simulated Annealing (Kirkpatrick et al., 1983), but two chess-flavored restrictions change its character. The temperature is scaled by the current fitness spread of the population, which makes the sacrifice budget self-calibrating across problems of wildly different magnitudes. And only minor pieces may be sacrificed: the King and the major pieces are never lost, so the elite of the population cannot be eroded by the acceptance rule.

2.6.3 En passant (adaptive local capture)

Once per iteration the King attempts an en-passant capture. Three trial points are drawn in a sparse Gaussian neighborhood,

$$\mathbf{Y}_m = \mathbf{K} + \sigma \circ \mathbf{z}_m \circ \mathbf{m}_m, \quad \mathbf{z}_m \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad m = 1, 2, 3, \quad (12)$$

where the random binary mask $\mathbf{m}_m$ activates each coordinate with probability $\max(0.2, 2/D)$. Any improving trial replaces the King. The radius σ follows a success rule of the kind long used in Evolution Strategies (Beyer & Schwefel, 2002): multiply by 1.3 after a successful capture, by 0.92 after a failure, and after 50 consecutive failures reset to $\sigma = 0.05 \mathbf{L} \max(a(t), 0.05)$ — a stalemate-avoidance restart. In our experience this mechanism is what keeps CA improving deep into the run, long after a fixed-radius local search would have gone quiet.

2.6.4 Castling

Every $c = 10$ iterations the King castles with the best Rook: a contiguous block of up to $D/4$ coordinates is copied from the Rook into a candidate King, and the exchange is kept only if it improves the King. It is a safeguarded structural jump, useful when good partial solutions have been discovered on different “files” of the board and need splicing together.

2.6.5 Threefold repetition (diversity preservation)

If an agent coincides with the King to numerical precision — something pinning can eventually cause — it is re-deployed uniformly in Ω , much as the threefold-repetition rule stops a game from going in circles. We did not add this rule for elegance; early prototypes taught us it is essential. Without it, pinning breeds exact clones of the King, every gap-proportional step length collapses to zero, and the search dies quietly in place.

2.6.6 Checkmate

The algorithm stops after T iterations. A stagnation-based early stop would be easy to add, but we keep it disabled in all experiments so that every algorithm consumes the same core budget of $N \times T$ population evaluations.

2.7 Parameter summary

Table 2 collects the base parameters and the single configuration used everywhere in this paper; several of them (pinning probability, the knight-leap rate, and others) are further modulated by the phase-conditioned schedule of Table 3 once the adaptive control layer of Section 2.11 is active — CA-static uses the values below unmodulated throughout. No per-problem tuning was done, for CA, for CA-static, or for any competitor.

Table 2: CA parameters and the fixed configuration used in all experiments.

Parameter	Symbol	Value
Role fractions (Queen / Rook / Bishop / Knight)	$\rho_q, \rho_r, \rho_b, \rho_n$	0.10/0.15/0.15/0.20
Development schedule	$a(t)$	$2(1 - t/T)$
Pinning probability / cap	$p_{\text{pin}}(t)$	$0.5 t/T$, at most 30% of coordinates
Sacrifice constant	θ_0	0.1
Castling period	c	10 iterations
En-passant probes per iteration	—	3
En-passant adaptation (success / failure / reset)	—	$\times 1.3$ / $\times 0.92$ / after 50 failures
Knight exploratory-leap distribution	—	uniform (Lévy, $\beta_\ell = 1.5$, optional)

2.8 Pseudocode

Algorithm: Chess Algorithm (CA)

Input: objective f , bounds $[l, u]$, dimension D ,
population N , iterations T

1 Initialize X and its board-mirror $l+u-X$; evaluate both; keep the


```

    fitter N (Eq. init); sort; King  $\leftarrow$  X(1); archive  $\leftarrow$  {}
2    $\leftarrow$  0.1 L; fails  $\leftarrow$  0; stall  $\leftarrow$  0; d_ema, acc_ema, eps_ema  $\leftarrow$  defaults
3   for t = 0 ... T-1:
4       a  $\leftarrow$  2(1 - t/T)
5       Read position: d_ema  $\leftarrow$  smoothed mean distance to King;
        phase  $\leftarrow$  Opening / Middlegame / Closed / Endgame (Eq. phase);
        if population idle (low acc_ema, eps_ema): halve a (Zugzwang)
6       Look up phase-conditioned rates: p_pin, leap mult., p_council,
        p_discovered-attack, p_interference, pawn gain (Table 3)
7       Assign roles by rank: Queens 10%, Rooks 15%, Bishops 15%,
        Knights 20%, Pawns rest
8       Move each piece by its operator; Knights fork or leap, Bishops
        occasionally interfere, Pawns advance at the phase's gain
9       Pin 30% of coordinates of each agent to King with prob. p_pin
10      Clip to bounds; evaluate F
11      Accept improvements; accept worse MINOR pieces with prob.  $e^{(-\Delta/)}$ ,
        set from the robust population spread (Eq. sacrifice)
12      En passant: 3 trials around King - masked Gaussian, plus a
        council or discovered-attack probe with prob. p_council /
        p_discovered-attack, plus a single-coordinate check probe;
        keep any improvement; adapt ; on success in Endgame, extend
        the winning displacement (windmill) while it keeps improving
13      Every 10 iters: Castle - block-swap King best Rook, keep if better
14      Threefold repetition: re-deploy agents identical to the King
15      Update King; re-rank population (pawn promotion); archive the
        King if it improved and is far from the current archive
16      if stall 25 (no significant improvement, Eq. fifty-move-rule):
        Pawn break (re-deploy all Pawns); King's march (6 long-radius
        probes, greedy-refine the best, keep if it improves the King);
        reheat the sacrifice budget  $\times 10$  for 10 iterations; replace the
        worst Queen with a random archived strongpoint; reset stall
17 return King

```

2.9 Flowchart

Figure 1 summarizes the core loop common to every iteration; the phase-dependent tactic selection and the blockade response of Section 2.11 sit inside its “apply role moves” and “update King” boxes respectively, and are given in full in the pseudocode above.

2.10 Properties and design rationale

Elitism and convergence. The King’s fitness never increases: it is replaced only by strictly better points, whether through the population update, en passant, or castling. That elitism buys the usual asymptotic guarantee (Proposition 2.1).

Proposition 2.1 (Elitist convergence in probability). *Let f be continuous on the compact box Ω with global minimum f^* . Under CA’s update rules, the best-so-far value $f(\mathbf{K}^{(t)})$ converges in probability to f^* as $T \rightarrow \infty$.*

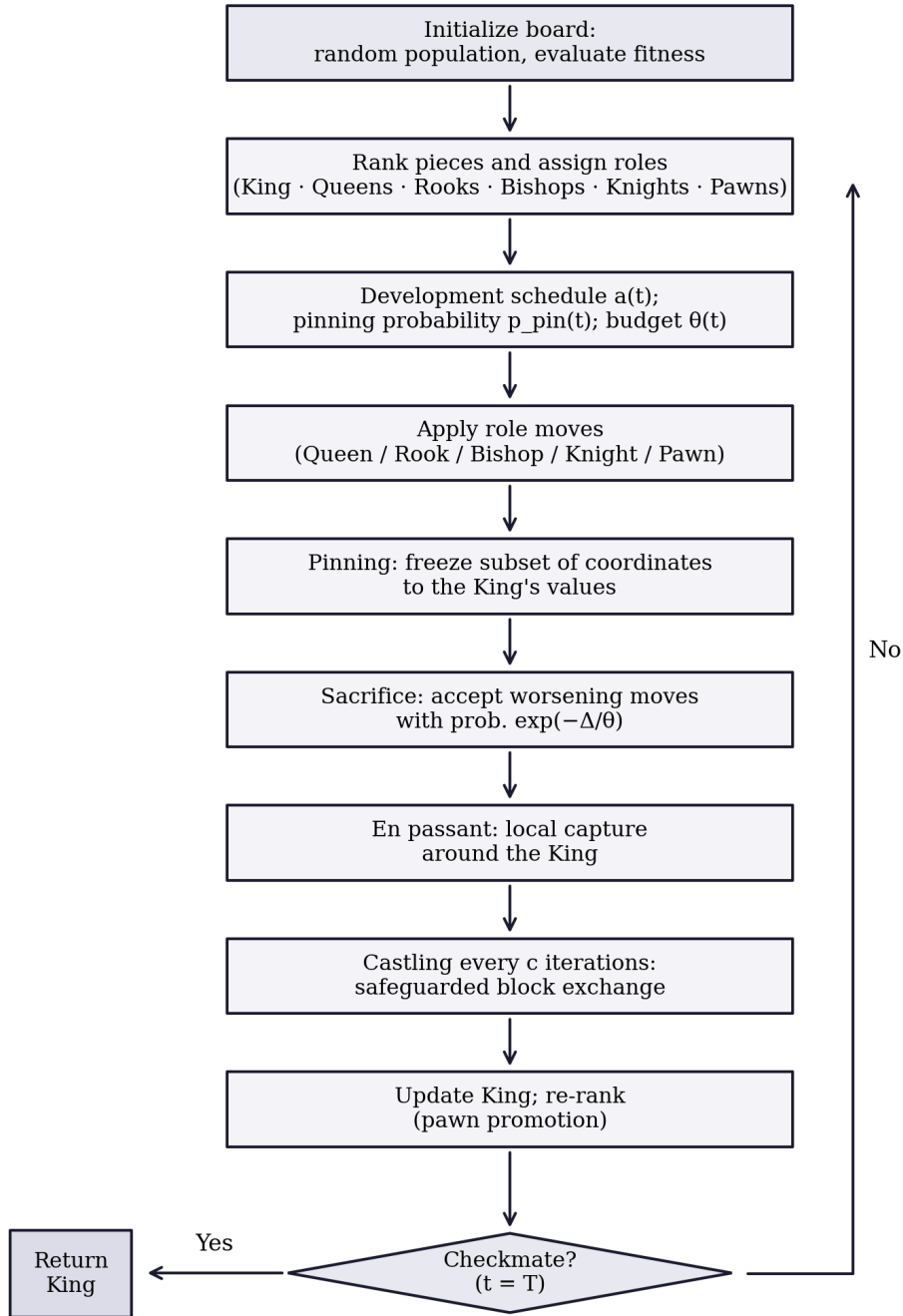


Figure 1: Flowchart of the Chess Algorithm.

Proof sketch. *The King sequence is elitist by construction. At every iteration the Knight’s exploratory leap and the threefold-repetition re-deployment both place uniform probability mass over Ω , so any open subset — in particular any neighborhood of a global minimizer, whose objective values come within ε of f^* by continuity — is visited with probability approaching one as iterations accumulate. Elitism plus this covering argument yields convergence in probability, exactly as in the standard analysis of elitist stochastic search.*

Of course, an asymptotic statement of this kind says nothing about what happens inside a budget of 15,000 evaluations. That question is empirical, and it is the subject of Section 3.

Exploration–exploitation balance. Exploration rests mainly on the Knights (Equation 8) and on the Queens early in the schedule (Equation 5); exploitation on the Pawns (Equation 10), on pinning, and on the en-passant success rule (Equation 12). The balance shifts smoothly through $a(t)$, $p_{\text{pin}}(t)$, and $\theta(t)$, and adapts to the state of the search through the gap-proportional step lengths and the spread-scaled sacrifice budget (Equation 11).

Parameter economy. Everything in this paper runs on the one configuration in Table 2. How sensitive performance is to the role fractions is a fair question we have not yet answered; it is on the future-work list rather than swept under the rug.

2.11 Adaptive tactical control

The mechanisms above give CA its movement vocabulary; what remains is to say how CA decides, moment to moment, which of them to lean on. A grandmaster does not run the same combination of tactics in a wide-open middlegame as in a locked pawn structure or a simplified endgame — which tactic is right depends on the position, not on a move counter. CA answers this with a lightweight state machine that reads four cheap statistics of the population each iteration and switches the active tactical repertoire accordingly, described below. For controlled comparison, every experiment in this paper is also run under **CA-static**: the identical operator set with this state machine disabled, so that role-appropriate tactics fire at fixed, non-adaptive rates instead. CA-static is not a separate algorithm; it is an ablation configuration, used throughout Section 4 and Section 5 to isolate exactly what the adaptive layer contributes.

2.11.1 Reading the position

Four exponentially-smoothed statistics ($\lambda = 0.8$) are recomputed every iteration. The population’s normalized divergence from the King,

$$d(t) = \frac{1}{L_{\text{span}}\sqrt{D}} \cdot \frac{1}{N-1} \sum_{i=2}^N \|\mathbf{X}_i^{(t)} - \mathbf{K}^{(t)}\|_2, \quad \tilde{d}(t) = 0.8\tilde{d}(t-1) + 0.2d(t), \quad (13)$$

answers “is the position open or closed?”, where $L_{\text{span}} = \max_j(u_j - l_j)$. Two further exponential moving averages track the initiative,

$$\alpha(t) = 0.8\alpha(t-1) + 0.2 \cdot \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\text{move}_i \text{ accepted}], \quad (14)$$

and the technical success rate of the King’s own local search,

$$\varepsilon(t) = \begin{cases} 0.8\varepsilon(t-1) + 0.2 & \text{en-passant capture succeeded at } t, \\ 0.8\varepsilon(t-1) & \text{otherwise.} \end{cases} \quad (15)$$

Finally, a stagnation counter borrows chess’s *fifty-move rule*: rather than reset on any improvement whatsoever, it resets only on a materially significant one, exactly as fifty moves without a pawn push or a capture — not merely without a game-theoretically perfect move — are needed to claim a draw:

$$s(t) = \begin{cases} 0, & f(\mathbf{K}^{(t)}) < f_{\text{ref}} - \max(10^{-4}|f_{\text{ref}}|, 10^{-10}), \\ s(t-1) + 1, & \text{otherwise,} \end{cases} \quad (16)$$

with f_{ref} updated to $f(\mathbf{K}^{(t)})$ whenever $s(t) = 0$. Early prototypes used a naive counter that reset on any improvement at all; because the en-passant rule of Section 2.6.3 makes microscopic refinements routinely, that counter never accumulated enough to trigger the blockade response of Section 2.11.4 even in runs that were visibly stuck in a secondary basin for hundreds of iterations. Equation Equation 16 is the fix.

2.11.2 Game phases

The smoothed divergence and the fraction of the budget elapsed, $\phi(t) = t/T$, jointly select one of four phases:

$$\text{phase}(t) = \begin{cases} \text{OPENING}, & \tilde{d}(t) > 0.22, \\ \text{MIDDLEGAME}, & 0.045 < \tilde{d}(t) \leq 0.22, \\ \text{CLOSED}, & \tilde{d}(t) \leq 0.045 \text{ and } \phi(t) \leq 0.5, \\ \text{ENDGAME}, & \tilde{d}(t) \leq 0.045 \text{ and } \phi(t) > 0.5. \end{cases} \quad (17)$$

The Closed phase answers a failure mode found during tuning: a population can collapse to a small radius around the King *early* in the search — a premature convergence, not a genuine endgame — and treating every low-divergence state as an endgame made this worse, since endgame tactics (heavy pinning, aggressive pawn advance) accelerate collapse rather than reversing it. Reading an early collapse as a **prematurely closed position** and answering with Nimzowitschian *prophylaxis* — restraint and reopening lines rather than immediate commitment — removes the pathology. Each phase activates a different parameter vector, summarized in Table 3.

Table 3: Phase-conditioned tactical schedule. Pinning probability p_{pin} , knight-leap probability multiplier, council probability p_{c} , discovered-attack probability p_{DA} , interference probability p_{intf} , and pawn step gain, by phase. *Endgame pinning is capped at 0.5.

Phase	p_{pin}	Leap mult.	p_{c}	p_{DA}	p_{intf}	Pawn gain
Opening	0	2.0	0	0	0.30	0.30
Middlegame	$0.5\phi(t)$	1.0	0.5	0	0.15	0.30
Closed	0	2.0	0.5	0.15	0.15	0.30
Endgame	$0.1+0.75\phi(t)^*$	0.5	0.7	0.15	0	0.45

One overlay applies regardless of phase. If the population has lost the initiative and the King’s local search has gone quiet at the same time,

$$\text{Zugzwang}(t) \iff \phi(t) > 0.2 \wedge \alpha(t) < 0.04 \wedge \varepsilon(t) < 0.05, \quad (18)$$

the development coefficient is halved for that one iteration, $a(t) \leftarrow a(t)/2$ — a *triangulation* move: change nothing structurally, spend a tempo, and let the next reading of the position be more informative before committing to a new phase.

2.11.3 New tactical operators

Novotny interference (Bishops). Named for the classical motif in which a piece is sacrificed on a square that blocks two enemy lines of defense at once, a Bishop is occasionally interposed on the segment between two distant, higher-ranked pieces rather than moving diagonally around the King:

$$\mathbf{X}'_i = \beta \mathbf{X}_a + (1 - \beta) \mathbf{X}_b + 0.02 a(t) \mathbf{L} \circ \mathbf{z}, \quad \beta \sim \mathcal{U}(0.3, 0.7), \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (19)$$

with a, b drawn without replacement from the fitter half of the population (probability p_{intf} , Table 3). Being a convex recombination of two arbitrary population members rather than a step along a coordinate axis, it is rotation-invariant, and it is the mechanism most directly responsible for CA’s gains on the CEC-2017 hybrid and composition functions (Section 4).

The knight’s fork. With probability 0.5, a Knight that does not take its exploratory leap instead plays

$$\mathbf{X}'_i = \mathbf{X}_i + \mathbf{r} \circ (\mathbf{K} - \mathbf{X}_i) + 0.8 (\mathbf{X}_a - \mathbf{X}_b), \quad a, b \sim \mathcal{U}\{1, \dots, i - 1\}, \quad \mathbf{r} \sim \mathcal{U}(0, 1)^D, \quad (20)$$

drifting toward the King while inheriting the differential structure of two better-ranked peers at once — attacking two targets simultaneously, in the spirit of the tactic that gives the move its name.

Discovered attack. An early version of this operator applied the geometry below to every Rook as a population move; tuning showed it tactically unsound at that scale, since the candidate is accepted by the ordinary improvement rule almost every time it is tried, collapsing the Rook sub-population onto the King–elite line within a handful of iterations. Restricted instead to a single, strictly elitist probe competing for one of the King’s own en-passant trials — active only in the Closed and Endgame phases —

$$\mathbf{Y}_{\text{DA}} = \mathbf{X}_e + u (\mathbf{K} - \mathbf{X}_e), \quad u \sim \mathcal{U}(1.2, 2.2), \quad e \sim \mathcal{U}\{1, 2, 3\}, \quad (21)$$

it recovers the intended effect — probing the far side of the incumbent along an elite-King line — without the pathology. The lesson generalizes: a tactic sound for a single elite piece can destabilize a whole role class, just as a discovered attack that wins material for one player is a blunder if every piece on the board tries it at once.

Royal council. Competing with the discovered attack for the same probe slot (mutually exclusive, selected by relative probability),

$$\mathbf{Y}_{\text{council}} = \mathbf{K} + 1.5 u (\mathbf{K} - \mathbf{c}), \quad \mathbf{c} = \frac{1}{2}(\mathbf{X}_{Q_1} + \mathbf{X}_{R_1}), \quad u \sim \mathcal{U}(0, 1), \quad (22)$$

reflects the King away from the midpoint of the best Queen and best Rook. Where isotropic Gaussian probes are blind to the shape of the elite, this reflection is population-shaped: when the elite lies strung along a narrow curved valley — the typical geometry along an active inequality constraint in the engineering problems of Section 5 — it points down the valley rather than across it. This single mechanism was responsible for closing most of the welded-beam gap observed in early prototypes.

Windmill. In the Endgame phase only, a successful en-passant or council capture with displacement $\delta = \mathbf{K}^{(t)} - \mathbf{K}_{\text{pre}}^{(t)}$ is extended,

$$\mathbf{K} \leftarrow \mathbf{K} + \delta \quad \text{while } f(\mathbf{K} + \delta) < f(\mathbf{K}), \quad \text{up to 3 repetitions,} \quad (23)$$

echoing the chess windmill’s repeating sequence of discovered checks, each one harvesting material.

Overprotection archive. Aron Nimzowitsch’s overprotection principle counsels defending a valuable point with more force than strictly necessary, so that pieces freed from other duties have a strong square to retreat to. CA archives up to five mutually distant past King positions,

$$\text{admit } \mathbf{K}^{(t)} \iff f(\mathbf{K}^{(t)}) < f(\mathbf{K}^{(t-1)}) \wedge \min_{\mathbf{a} \in \mathcal{A}} \frac{\|\mathbf{K}^{(t)} - \mathbf{a}\|}{L_{\text{span}} \sqrt{D}} > 0.15, \quad (24)$$

evicting the worst by fitness on overflow. These strongpoints are the fallback used by the blockade response below.

2.11.4 Blockade: pawn break, King’s march, and Tal’s speculative sacrifice

When the fifty-move counter reaches $s(t) \geq 25$ (Equation 16), three mechanisms fire together. All Pawns are re-deployed uniformly at random — a *pawn break*, opening lines in a closed position. The King probes six long-radius candidates at three scales,

$$\mathbf{Y}_m = \mathbf{K} + \rho_m \mathbf{L} \circ \mathbf{z}_m, \quad \rho_m \in \{0.1, 0.1, 0.2, 0.2, 0.4, 0.4\}, \quad \mathbf{z}_m \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (25)$$

then *consolidates* the best candidate with an eight-step greedy local refinement of shrinking radius before comparing it against the incumbent King. The order is deliberate: refine-then-compare lets the march accept a basin whose raw landing point is worse than the incumbent but whose refined optimum is better, while the final elitist comparison still guarantees the King’s fitness sequence never worsens. Simultaneously, the sacrifice budget of Equation 11 is reheated tenfold for the next ten iterations — a nod to Mikhail Tal’s willingness to give up material for an initiative he could not yet fully calculate, rather than keep playing safe moves that have already stopped producing progress — and, if the archive of Equation 24 is non-empty, the worst-ranked Queen is replaced by a random archived strongpoint, giving the population a proven foothold rather than a blind restart.

2.11.5 Opening principle: opposition-based initialization

The initial population is evaluated together with its reflection through the board’s center, $\mathbf{X}_{\text{mirror}}^{(0)} = \mathbf{l} + \mathbf{u} - \mathbf{X}^{(0)}$, and the fitter half of the combined $2N$ candidates is kept. We report this honestly rather than dress it up: isolating this mechanism during tuning found a *mixed* effect — better on two of four micro-benchmark problems tested, worse on the other two — not a consistent gain. It is kept by default because it never produced a large regression and costs only one extra population evaluation, paid once; a dedicated ablation across the full CEC-2017 suite is future work.

3 Benchmark Experiments

3.1 Experimental protocol

CA is compared against its own non-adaptive ablation, CA-static (Section 2.11), and four established metaheuristics: a real-coded GA with tournament selection, BLX- α crossover, Gaussian mutation, and elitism (Goldberg, 1989; Holland, 1975); PSO with linearly decreasing inertia (Kennedy & Eberhart, 1995); SA (Kirkpatrick et al., 1983), granted a population-equivalent evaluation budget so the comparison stays fair; and GWO (Mirjalili et al., 2014).

The suite consists of six standard functions covering unimodal, ill-conditioned, and highly multimodal terrain (Table 4):

Table 4: Benchmark functions ($D = 30$).

Function	Type	Search domain	f^*
F1 Sphere	Unimodal	$[-100, 100]^{30}$	0
F2 Rosenbrock	Unimodal, ill-conditioned valley	$[-30, 30]^{30}$	0
F3 Rastrigin	Multimodal	$[-5.12, 5.12]^{30}$	0
F4 Griewank	Multimodal	$[-600, 600]^{30}$	0
F5 Ackley	Multimodal	$[-32, 32]^{30}$	0
F6 Schwefel 2.22	Unimodal, non-separable norm	$[-10, 10]^{30}$	0

Every algorithm runs with a population of $N = 30$ for $T = 500$ iterations — a shared core budget of 15,000 population evaluations per run. In the interest of full transparency: CA additionally spends three en-passant probes per iteration and one castling probe every ten iterations (Section 2), roughly 1,550 extra evaluations, or about ten percent. The gaps reported below span orders of magnitude, so this overhead cannot be what drives them; note also that GWO beats CA *despite* it. Each algorithm–function pair is repeated over 30 independent runs, and all algorithms share the same seed at a given run index, so everyone faces the same initial conditions. No parameters were tuned per problem for any method.

Statistics follow Derrac et al. (2011): one-way ANOVA to establish that the algorithm factor matters at all, then pairwise two-sided Wilcoxon rank-sum tests (CA against each competitor) at $\alpha = 0.05$, which make no normality assumption about the final-error distributions.

3.2 Descriptive results

Table 5 reports the mean, standard deviation, best, and worst final errors for every algorithm–function pair.

Table 5: Mean, standard deviation, best, and worst final objective values over 30 runs (best mean per function in bold).

Function	Algo- rithm	Mean	Std	Best	Worst
Sphere	CA	6.159e-05	1.152e-04	2.149e-07	4.760e-04
Sphere	CA-static	4.630e-06	4.253e-06	2.111e-07	1.572e-05
Sphere	GA	9.475e-02	5.399e-02	2.890e-02	2.946e-01
Sphere	PSO	8.455e-01	7.216e-01	1.194e-01	3.131e+00
Sphere	SA	1.013e+04	2.218e+03	6.628e+03	1.612e+04
Sphere	GWO	4.678e-31	8.194e-31	6.431e-33	3.925e-30
Rosenbrock	CA	7.756e+01	4.383e+01	2.431e+01	1.912e+02
Rosenbrock	CA-static	5.513e+01	3.969e+01	7.695e+00	1.414e+02
Rosenbrock	GA	1.056e+02	4.633e+01	2.895e+01	2.863e+02
Rosenbrock	PSO	6.445e+03	2.238e+04	8.896e+01	9.021e+04
Rosenbrock	SA	2.185e+07	5.693e+06	7.615e+06	3.486e+07
Rosenbrock	GWO	2.683e+01	6.174e-01	2.592e+01	2.796e+01
Rastrigin	CA	1.953e+01	8.020e+00	7.960e+00	4.477e+01
Rastrigin	CA-static	1.924e+01	4.897e+00	9.951e+00	3.383e+01
Rastrigin	GA	1.954e+01	6.823e+00	1.008e+01	4.187e+01
Rastrigin	PSO	5.971e+01	1.484e+01	3.306e+01	9.545e+01
Rastrigin	SA	2.105e+02	2.951e+01	1.598e+02	2.553e+02
Rastrigin	GWO	1.023e+01	1.078e+01	5.684e-14	4.680e+01
Griewank	CA	8.177e-03	7.777e-03	2.282e-05	2.473e-02
Griewank	CA-static	1.123e-02	1.143e-02	1.617e-05	3.705e-02
Griewank	GA	2.816e-01	7.971e-02	1.217e-01	4.364e-01
Griewank	PSO	6.164e-01	2.367e-01	2.902e-02	1.025e+00
Griewank	SA	9.190e+01	2.324e+01	4.826e+01	1.563e+02
Griewank	GWO	4.303e-03	9.094e-03	0.000e+00	3.591e-02
Ackley	CA	6.049e-02	3.189e-01	1.561e-04	1.778e+00
Ackley	CA-static	3.902e-02	2.073e-01	1.508e-04	1.155e+00
Ackley	GA	7.757e-02	1.486e-02	4.767e-02	1.099e-01
Ackley	PSO	1.045e+00	5.491e-01	8.837e-02	2.341e+00
Ackley	SA	4.483e+00	3.618e-01	3.807e+00	5.247e+00
Ackley	GWO	3.325e-14	4.369e-15	2.176e-14	4.308e-14
Schwefel 2.22	CA	1.554e-03	6.928e-04	3.858e-04	3.011e-03
Schwefel 2.22	CA-static	8.415e-04	3.824e-04	3.074e-04	1.562e-03
Schwefel 2.22	GA	7.438e-02	1.815e-02	4.451e-02	1.138e-01
Schwefel 2.22	PSO	7.941e-01	2.486e+00	3.962e-02	1.010e+01
Schwefel 2.22	SA	1.121e+07	2.877e+07	1.033e+03	1.180e+08
Schwefel 2.22	GWO	9.928e-19	9.462e-19	1.034e-19	4.919e-18

3.3 Convergence behavior

Figure 2 shows the median best-so-far curves, and three patterns are worth pointing out. On F1, F4, and F6, CA keeps descending through the whole run — the en-passant success rule goes on finding improvements deep in the endgame, where GA and PSO stagnated hundreds of iterations earlier. On the ill-conditioned Rosenbrock valley (F2), every population method slows to a crawl, CA and GA tracking each other closely the whole way down. CA and CA-static track each other almost exactly on every function, visually confirming the near-total absence of a significant difference between them on this suite. And GWO shows its well-documented strength on this classical suite, converging fastest and deepest throughout. The distributions of final values behind these curves are shown in Figure 3.

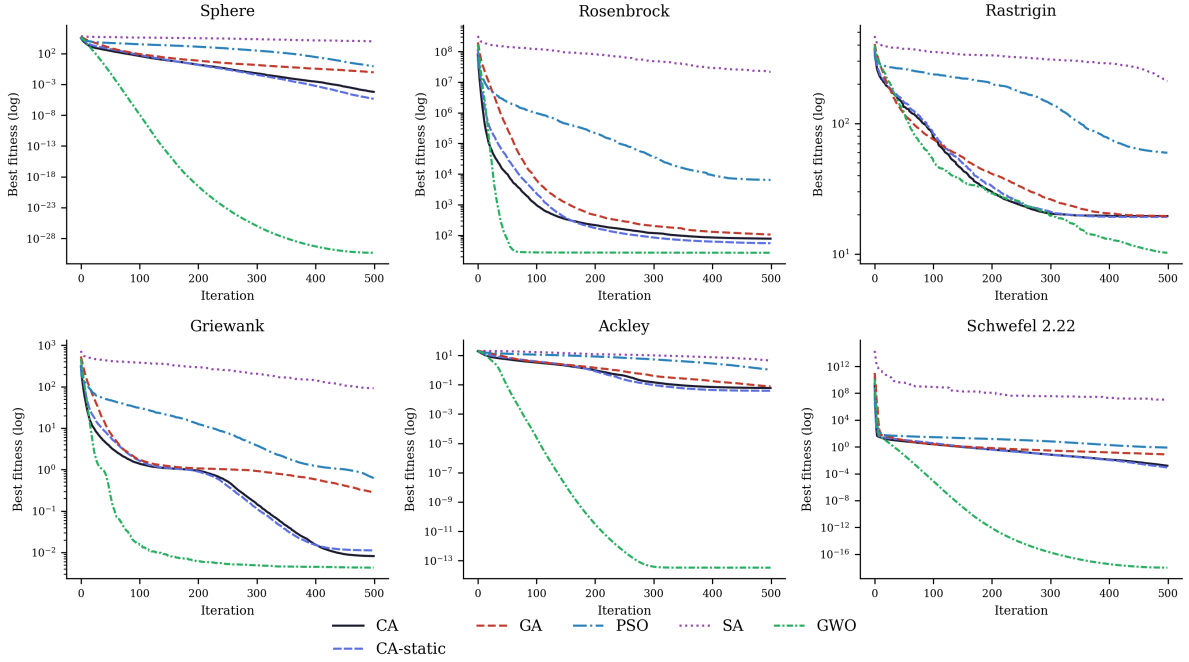


Figure 2: Median convergence curves (30 runs, semi-log scale).

3.4 Statistical tests

One-way ANOVA confirms that the algorithm factor is highly significant on every function (Table 6):

Table 6: One-way ANOVA over the six algorithms, per function.

Function	F-statistic	p-value
Sphere	605.079	5.569e-108
Rosenbrock	427.018	1.130e-95
Rastrigin	777.692	5.369e-117
Griewank	451.683	1.229e-97
Ackley	951.036	2.680e-124
Schwefel 2.22	4.403	8.433e-04

The pairwise Wilcoxon tests give the sharper picture (Table 7):

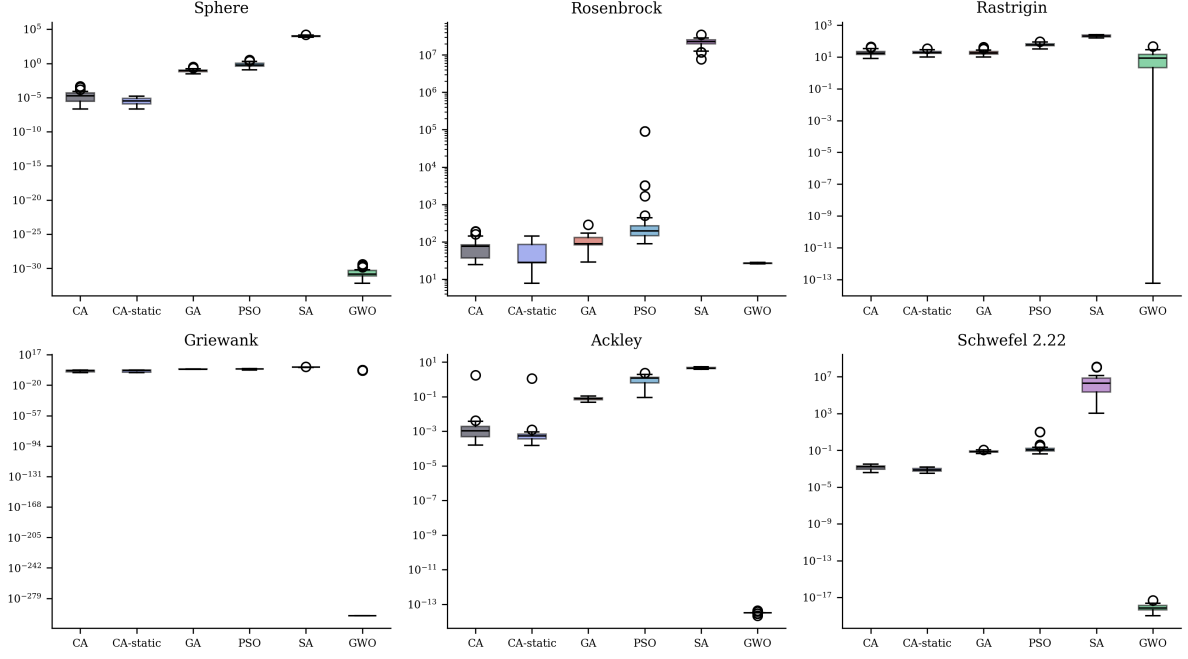


Figure 3: Distribution of final objective values over 30 runs.

Table 7: Wilcoxon rank-sum tests, CA versus each competitor ($\alpha = 0.05$).

Function	Comparison	p-value	Result ($\alpha = 0.05$)
Sphere	CA vs CA-static	1.480e-03	CA worse
Sphere	CA vs GA	2.872e-11	CA better
Sphere	CA vs PSO	2.872e-11	CA better
Sphere	CA vs SA	2.872e-11	CA better
Sphere	CA vs GWO	2.872e-11	CA worse
Rosenbrock	CA vs CA-static	6.043e-02	not significant
Rosenbrock	CA vs GA	1.086e-03	CA better
Rosenbrock	CA vs PSO	2.552e-09	CA better
Rosenbrock	CA vs SA	2.872e-11	CA better
Rosenbrock	CA vs GWO	2.954e-08	CA worse
Rastrigin	CA vs CA-static	6.681e-01	not significant
Rastrigin	CA vs GA	5.154e-01	not significant
Rastrigin	CA vs PSO	6.373e-11	CA better
Rastrigin	CA vs SA	2.872e-11	CA better
Rastrigin	CA vs GWO	6.160e-05	CA worse
Griewank	CA vs CA-static	7.007e-01	not significant
Griewank	CA vs GA	2.872e-11	CA better
Griewank	CA vs PSO	2.872e-11	CA better
Griewank	CA vs SA	2.872e-11	CA better
Griewank	CA vs GWO	1.479e-05	CA worse
Ackley	CA vs CA-static	1.099e-02	CA worse
Ackley	CA vs GA	5.317e-10	CA better
Ackley	CA vs PSO	4.403e-10	CA better
Ackley	CA vs SA	2.872e-11	CA better
Ackley	CA vs GWO	2.872e-11	CA worse
Schwefel 2.22	CA vs CA-static	5.101e-05	CA worse

Function	Comparison	p-value	Result ($\alpha = 0.05$)
Schwefel 2.22	CA vs GA	2.872e-11	CA better
Schwefel 2.22	CA vs PSO	2.872e-11	CA better
Schwefel 2.22	CA vs SA	2.872e-11	CA better
Schwefel 2.22	CA vs GWO	2.872e-11	CA worse

3.5 Discussion

Read together, the tables support a differentiated verdict rather than a slogan. Against PSO and SA, CA is significantly better on all six functions. Against GA it wins five of six — on Sphere, Rosenbrock, Griewank, Ackley, and Schwefel 2.22 — while Rastrigin ends in a statistical tie. Against GWO the story reverses: GWO is significantly better on all six classical benchmarks, consistent with the aggressive exploitation it is known for on exactly this suite (Mirjalili et al., 2014). We see no point in arguing with that result. On standard unconstrained test functions, CA does not overtake GWO, full stop.

The comparison against CA-static is the one worth dwelling on, because it runs against the paper’s own expectations. On this classical, low-structure suite, disabling the adaptive layer of Section 2.11 never hurts and sometimes *helps*: CA-static is significantly better than CA on Sphere, Ackley, and Schwefel 2.22, and statistically tied on the other three, with not a single function where the adaptive layer wins outright. We read this as an honest and informative negative result rather than something to bury: the phase-reading machinery is built to recognize open, closed, and endgame *structure*, and a smooth unimodal or mildly multimodal function in a fixed box offers it little of that structure to exploit — the state machine spends its overhead reading a position that does not change much. The CEC-2017 and engineering-design landscapes of Section 4 and Section 5 are exactly where that overhead should start paying for itself, and whether it does is the empirical question those sections answer.

What should a practitioner take from this? The rankings are problem-dependent — precisely the situation Wolpert & Macready (1997) predicts — so the interesting question is not whether CA wins on Sphere but whether it holds up on a harder, standardized synthetic suite and on structured engineering problems. That is where we go next: Section 4 extends the comparison to the full CEC-2017 suite, Section 5 turns to constrained engineering design, and Section 6 returns to the transportation motivation that opened the paper — and the ranking does change.

4 The Full CEC-2017 Benchmark Suite

4.1 Motivation and roster

The six-function suite of Section 3 is standard but small, and by the standards this literature has settled on since — including our own baseline, GWO (Mirjalili et al., 2014) — a serious claim of competitiveness is expected to survive a larger, harder, and independently curated suite. CEC-2017 supplies exactly that: 29 functions, shifted, rotated, and in many cases built by hybridizing or composing several base functions so that an algorithm cannot exploit coordinate-axis alignment or a single basin shape. We evaluate CA (Section 2.11) and its non-adaptive ablation CA-static against GWO, PSO, GA (our own implementations from Section 3), and WOA (Mirjalili & Lewis, 2016) (third-party, `mealpy` defaults, as in Section 7), at $D = 30$ with population 30, 500 iterations, and 30 independent runs per algorithm–function pair, run r of

every algorithm sharing seed $\text{SEED}_0 + r$ on the unit hypercube (real bounds folded into the decoder, as throughout this paper).

4.2 Data integrity: auditing the reference implementation

CEC-2017 was accessed through `opfunu` v1.0.1, a third-party Python port of the official suite, and two independent checks were applied before any result was trusted. First, every candidate function was evaluated at the library’s own reported global optimizer $\mathbf{x}^*$, requiring $|f(\mathbf{x}^*) - f^*| \leq 10^{-6} \max(1, |f^*|)$; this is a necessary but not sufficient check, since it verifies the function only at one point and says nothing about the rest of the domain. Second, we required the landscape to be discriminative: F5 (Shifted-and-Rotated Schaffer F7) passed the first check exactly ($f(\mathbf{x}^*) = f^* = 500.0$) but random samples spanning the entire search box returned values between 500.15 and 500.55 — a surface so nearly flat that no optimizer, ours or any competitor’s, could extract a meaningful gradient from it — and was excluded on that basis.

A function can pass both of those checks and still be broken elsewhere in the domain. Four further functions were caught only empirically, during optimization itself, because every algorithm in the roster — not CA specifically — returned a best-of-30-runs value definitionally impossible under a correct f^* :

Table 8: Post-hoc integrity audit: functions where at least one algorithm’s best run fell below the library’s claimed global optimum, evidence of an implementation defect rather than an artifact of any one algorithm’s search behavior.

Function	opfunu name	Claimed f^*	Min. raw value observed
F9	Shifted and Rotated Schwefel	900.0	-2491.7
F15	Hybrid Function 6	1500.0	865.3
F19	Hybrid Function 10	1900.0	1470.7
F21	Composition Function 2	2100.0	-7492.4

F21 is the most severe case: essentially every algorithm in the roster, including the weakest, found points the library’s own optimum could not explain. All five functions (F5, F9, F15, F19, F21) are excluded from the analysis below, leaving **24 validated functions**; the full pre-audit numbers are kept on record in the repository for transparency rather than quietly dropped.

4.3 Results

Table 9 reports the mean Friedman rank of each algorithm across the 24 functions, and Table 10 the win/tie/loss record of CA against each competitor under a two-sided Wilcoxon rank-sum test at $\alpha = 0.05$. The full per-function statistics are in Table 11; Figure 4 shows median convergence on six representative functions spanning unimodal, multimodal, hybrid, and composition landscapes.

Table 9: Mean Friedman rank across 24 validated CEC-2017 functions (lower is better; $\chi^2 = 87.40$, $p = 2.4 \times 10^{-17}$).

Algorithm	Mean Friedman rank
CA	2.000
GA	2.167

Algorithm	Mean Friedman rank
CA-static	2.208
PSO	4.375
GWO	4.417
WOA	5.833

Table 10: CA win/tie/loss against each competitor, CEC-2017, 24 functions, Wilcoxon rank-sum at $\alpha = 0.05$.

Competitor	Win	Tie	Loss
CA-static	4	18	2
GWO	23	1	0
PSO	21	3	0
GA	7	7	10
WOA	24	0	0

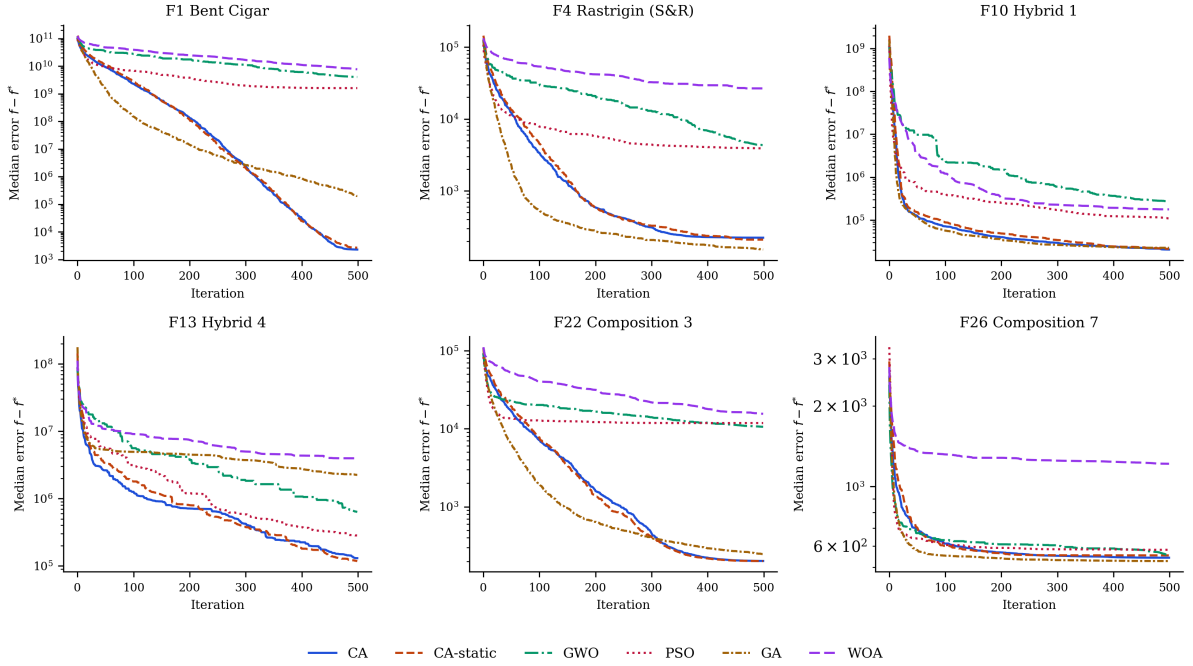


Figure 4: Median convergence on six representative CEC-2017 functions (30 runs, semi-log scale).

Table 11: Mean, standard deviation, best, and worst final error $f - f^*$ over 30 runs, all 24 validated CEC-2017 functions (best mean per function in bold).

Func	Algorithm	Mean	Std	Best	Worst
F1	CA	3011	2587	197.2	1.223e+04
F1	CA-static	4186	4081	190.5	1.709e+04
F1	GWO	4.3781e+09	3.6738e+09	1.2942e+07	1.4000e+10
F1	PSO	2.3283e+09	2.5019e+09	6.4555e+05	9.9419e+09
F1	GA	2.4003e+05	1.4751e+05	2.947e+04	6.9702e+05

Func	Algorithm	Mean	Std	Best	Worst
F1	WOA	8.4880e+09	4.3850e+09	2.2548e+09	2.3894e+10
F2	CA	107.7	204.6	0.06075	780.8
F2	CA-static	3.805	6.16	0.02656	24.72
F2	GWO	1.564e+04	1.178e+04	2454	4.541e+04
F2	PSO	2742	5215	113.1	2.82e+04
F2	GA	10.85	5.971	2.576	26.3
F2	WOA	5.175e+04	1.826e+04	2.045e+04	8.45e+04
F3	CA	107.8	43.4	22.7	198.6
F3	CA-static	99.03	39.97	16.04	177.3
F3	GWO	411.1	447.5	42.44	1871
F3	PSO	310.5	445.3	35.29	2096
F3	GA	86.67	50.25	31.86	202.1
F3	WOA	1416	750.8	479.5	3244
F4	CA	228.9	42.15	133.3	327.4
F4	CA-static	212.5	55.14	121.7	391
F4	GWO	6084	4170	688.8	1.571e+04
F4	PSO	4978	3476	1089	1.357e+04
F4	GA	148	35.3	87.11	219.2
F4	WOA	2.64e+04	8785	1.119e+04	5.035e+04
F6	CA	273.9	75.94	149.4	428.3
F6	CA-static	231.5	47.18	153.4	324.8
F6	GWO	8.224e+04	1.1049e+05	1.011e+04	5.2492e+05
F6	PSO	1.9461e+05	1.7735e+05	2.719e+04	8.9813e+05
F6	GA	312.4	52.84	229.3	412.1
F6	WOA	3.7174e+05	1.5982e+05	1.4091e+05	9.1497e+05
F7	CA	207.8	35.02	124	282
F7	CA-static	215.5	44.72	124	298
F7	GWO	377	140.5	111.9	604.1
F7	PSO	255	77.89	125.4	413
F7	GA	251.2	66.57	118	358
F7	WOA	846.4	130.5	649.5	1142
F8	CA	4.821	2.025	2.004	12.04
F8	CA-static	5.035	2.34	0.9982	9.215
F8	GWO	6.806	4.302	0.8538	19.03
F8	PSO	4.802	2.868	0.1875	10.58
F8	GA	7.657	3.671	1.908	18.69
F8	WOA	25.43	10.2	11.36	43.89
F10	CA	2.232e+04	9882	8453	5.258e+04
F10	CA-static	2.353e+04	1.528e+04	2787	6.693e+04
F10	GWO	2.9010e+05	7.887e+04	1.5099e+05	4.7707e+05
F10	PSO	3.4876e+05	8.7568e+05	1.247e+04	3.7428e+06
F10	GA	2.328e+04	1.348e+04	3055	6.502e+04
F10	WOA	3.4194e+05	7.9388e+05	6.97e+04	4.5950e+06
F11	CA	3.2870e+05	2.6355e+05	3.439e+04	1.0288e+06
F11	CA-static	5.8859e+05	6.3353e+05	3.506e+04	2.6364e+06
F11	GWO	1.3921e+08	2.6493e+08	4.3572e+06	1.1956e+09
F11	PSO	2.2279e+08	4.7394e+08	8.507e+04	2.2180e+09
F11	GA	5.292e+04	3.32e+04	1.273e+04	1.7377e+05
F11	WOA	4.7012e+08	7.0769e+08	2.6239e+07	3.1975e+09

Func	Algorithm	Mean	Std	Best	Worst
F12	CA	2.525e+04	4.452e+04	3121	2.3720e+05
F12	CA-static	3.129e+04	4.555e+04	5391	2.5447e+05
F12	GWO	1.0450e+08	4.9388e+08	6.1448e+05	2.7589e+09
F12	PSO	1.7909e+08	5.0647e+08	3.127e+04	2.6839e+09
F12	GA	8973	6242	755.8	2.537e+04
F12	WOA	1.7695e+08	4.6631e+08	9.4148e+06	2.6417e+09
F13	CA	2.2120e+05	1.8679e+05	1.834e+04	6.6846e+05
F13	CA-static	1.9711e+05	1.7575e+05	3.497e+04	7.7942e+05
F13	GWO	1.7826e+06	2.4331e+06	2.0212e+05	1.0381e+07
F13	PSO	4.5379e+05	4.2879e+05	9.481e+04	2.0864e+06
F13	GA	2.8917e+06	2.4925e+06	4.78e+04	8.3697e+06
F13	WOA	5.2760e+06	4.6962e+06	2.4167e+05	1.5368e+07
F14	CA	2.36e+04	7888	6264	4.497e+04
F14	CA-static	3.082e+04	1.062e+04	1.202e+04	5.938e+04
F14	GWO	1.7884e+07	3.8538e+07	3.0152e+05	1.8339e+08
F14	PSO	1.0135e+07	4.1026e+07	1.687e+04	2.2348e+08
F14	GA	2.169e+04	1.141e+04	3974	5.219e+04
F14	WOA	5.6631e+07	8.3635e+07	2.8791e+05	2.8611e+08
F16	CA	1549	432.1	666.2	2333
F16	CA-static	1338	524.7	505.6	2708
F16	GWO	1.3404e+05	5.821e+04	5.668e+04	2.4905e+05
F16	PSO	5.557e+04	4.634e+04	1704	1.8705e+05
F16	GA	1184	518.8	325.6	2513
F16	WOA	1.4850e+09	7.9939e+09	3.722e+04	4.4533e+10
F17	CA	5.881e+04	4.279e+04	1.717e+04	2.3831e+05
F17	CA-static	4.293e+04	1.876e+04	1.042e+04	1.0299e+05
F17	GWO	1.3832e+05	5.843e+04	7.511e+04	3.9912e+05
F17	PSO	3.8353e+05	1.0266e+06	6.179e+04	5.6591e+06
F17	GA	7.1096e+05	7.7840e+05	1.339e+04	3.2841e+06
F17	WOA	1.8559e+05	3.4863e+05	3.658e+04	2.0331e+06
F18	CA	1.463e+04	1.355e+04	251.5	5.897e+04
F18	CA-static	1.025e+04	1.217e+04	491.6	4.737e+04
F18	GWO	5.2931e+09	1.0298e+10	5.402e+04	5.0407e+10
F18	PSO	6.0458e+09	1.7771e+10	1.222e+04	8.6826e+10
F18	GA	2.014e+04	1.389e+04	4736	6.583e+04
F18	WOA	9.9471e+09	2.0025e+10	2.4809e+07	9.9822e+10
F20	CA	363.1	175.6	100.1	694.9
F20	CA-static	291.4	184.7	100.1	597.7
F20	GWO	4150	5185	583.2	2.272e+04
F20	PSO	2035	1133	400.9	5567
F20	GA	287.8	107.8	110.5	396.2
F20	WOA	1.446e+04	1.082e+04	2128	3.639e+04
F22	CA	314.5	210.5	200.6	853.4
F22	CA-static	317.3	199.2	100.4	826.1
F22	GWO	9891	4098	1937	1.628e+04
F22	PSO	1.08e+04	5061	1584	2.012e+04
F22	GA	265.7	66.79	228.3	515.4
F22	WOA	1.72e+04	8057	6458	4.191e+04
F23	CA	224.5	94.6	200.3	676.9

Func	Algorithm	Mean	Std	Best	Worst
F23	CA-static	272	150.3	200.3	720.6
F23	GWO	5984	2107	1626	9470
F23	PSO	5347	2684	559	9612
F23	GA	254.3	78.26	220.7	548.5
F23	WOA	1.296e+04	8319	4099	3.544e+04
F24	CA	451.7	22.39	428.2	531.4
F24	CA-static	443.2	18.05	424.2	501.8
F24	GWO	639.5	214.1	436.6	1614
F24	PSO	490	127.5	421.9	1067
F24	GA	435.7	18.28	422.5	502
F24	WOA	1038	314.7	665.6	2286
F25	CA	866.9	45.29	788.2	1084
F25	CA-static	871.5	16.87	835.8	903.7
F25	GWO	913.7	30.28	847.7	980.6
F25	PSO	1052	145.2	865.3	1509
F25	GA	840.1	14.75	793.9	870.7
F25	WOA	2731	2035	912	1.034e+04
F26	CA	547.1	31.32	500.7	600.7
F26	CA-static	558.8	49.5	473.4	681.5
F26	GWO	558.8	28.63	511.8	650.4
F26	PSO	588.6	58.75	501.8	786.7
F26	GA	523	22.3	470.9	563.9
F26	WOA	1231	240.7	830.8	1817
F27	CA	227.6	162.1	44.06	482.2
F27	CA-static	229.1	172.3	24.44	477.4
F27	GWO	609.3	182.8	475.7	1377
F27	PSO	810.5	322.4	275.2	1475
F27	GA	102.6	145.3	18.38	476.1
F27	WOA	1073	666.2	562.9	2837
F28	CA	9.86e+04	1.3873e+05	1.587e+04	7.2781e+05
F28	CA-static	1.2912e+05	1.9055e+05	1.372e+04	8.3153e+05
F28	GWO	2.6902e+08	4.3159e+08	7.957e+04	1.8853e+09
F28	PSO	3.0455e+07	6.4775e+07	2.427e+04	2.8840e+08
F28	GA	1.4133e+05	2.3724e+05	3842	1.1699e+06
F28	WOA	8.6269e+08	1.0617e+09	1.7668e+07	4.1723e+09
F29	CA	8.8e+04	1.2333e+05	1.973e+04	6.9350e+05
F29	CA-static	2.2181e+05	2.5501e+05	2.74e+04	1.0666e+06
F29	GWO	5.0197e+08	1.1373e+09	1.4198e+05	4.2573e+09
F29	PSO	6.3055e+08	1.9914e+09	2.16e+04	9.3748e+09
F29	GA	6.5429e+05	1.3877e+06	8067	5.6345e+06
F29	WOA	3.1907e+09	5.4029e+09	5.7569e+06	2.3284e+10

4.4 Discussion

CA takes the best mean Friedman rank of the six algorithms compared (2.000), narrowly ahead of GA (2.167) and CA-static (2.208), and well ahead of PSO (4.375), GWO (4.417), and WOA (5.833). Against GWO, PSO, and WOA the record is close to a sweep — 23/24, 21/24, and 24/24 wins respectively, with not a single loss to any of the three. Against its own non-adaptive

ablation, CA-static, the adaptive machinery of Section 2.11 wins 4 and loses 2 of 24, with 18 ties: a real but modest net gain, which is the expected signature of a well-tuned control layer added on top of an already-competitive base algorithm rather than a wholesale replacement of it.

The one comparison that does not flatter CA is against GA, where the near-tied mean rank conceals a 7-win / 7-tie / 10-loss record function by function. The ten losses are not scattered at random: they are F4, F11, F12, F16, F20, F22, and F24–F27 — Rastrigin-family, hybrid, and composition landscapes whose basins are separated by distances comparable to the search domain itself. GA’s BLX- α crossover routinely proposes offspring *outside* the interval spanned by two parents, an operator that can relocate a candidate across the entire domain in a single step. Every CA operator introduced in Section 2.11, by contrast, is anchored to the King, an elite peer, or a bounded local neighborhood; even the Knight’s exploratory leap resamples only two of thirty coordinates. This is not a flaw to be argued away so much as a clean illustration of the No-Free-Lunch theorem (Wolpert & Macready, 1997): CA was tuned specifically against constrained engineering geometry and smooth hybrid/composition exploitation (Section 2.11), and that specialization has a measurable opportunity cost on landscapes whose defining difficulty is long-range, unstructured basin-hopping. We return to this trade-off, and to a concrete proposal for closing it, in Section 8.

5 Constrained Engineering Design Benchmarks

5.1 Problems and roster

Alongside standardized synthetic suites, it is standard practice in this literature to validate a new metaheuristic on the small set of constrained engineering design problems that recur across dozens of published comparisons — precisely because a shared, literature-anchored problem lets a reader compare our numbers against results we did not produce ourselves. We use seven such problems in their commonly published formulations: welded beam design, tension/compression spring design, pressure vessel design, speed reducer design, three-bar truss design, gear train design, and cantilever beam design. Constraints are handled by the same static-penalty approach used elsewhere in this paper (Section 7): $f_{\text{pen}}(\mathbf{x}) = f(\mathbf{x}) + 10^6 \sum_i \max(0, g_i(\mathbf{x}))^2$. Roster and protocol are identical to Section 4 (CA, CA-static, GWO, PSO, GA, WOA; population 30, 500 iterations, 30 runs).

5.2 Results

Table 12 reports the mean Friedman rank of each algorithm across the seven problems, Table 13 the win/tie/loss record of CA against each competitor, and Table 14 the full per-problem statistics; Figure 5 shows median convergence on all seven.

Table 12: Mean Friedman rank across the 7 engineering design problems (lower is better; $\chi^2 = 17.61$, $p = 0.0035$).

Algorithm	Mean Friedman rank
CA	2.000
CA-static	2.286
GWO	3.143
PSO	3.714

Algorithm	Mean Friedman rank
GA	4.286
WOA	5.571

Table 13: CA win/tie/loss against each competitor, engineering design problems, Wilcoxon rank-sum at $\alpha = 0.05$.

Competitor	Win	Tie	Loss
CA-static	1	6	0
GWO	3	4	0
PSO	4	2	1
GA	6	1	0
WOA	7	0	0

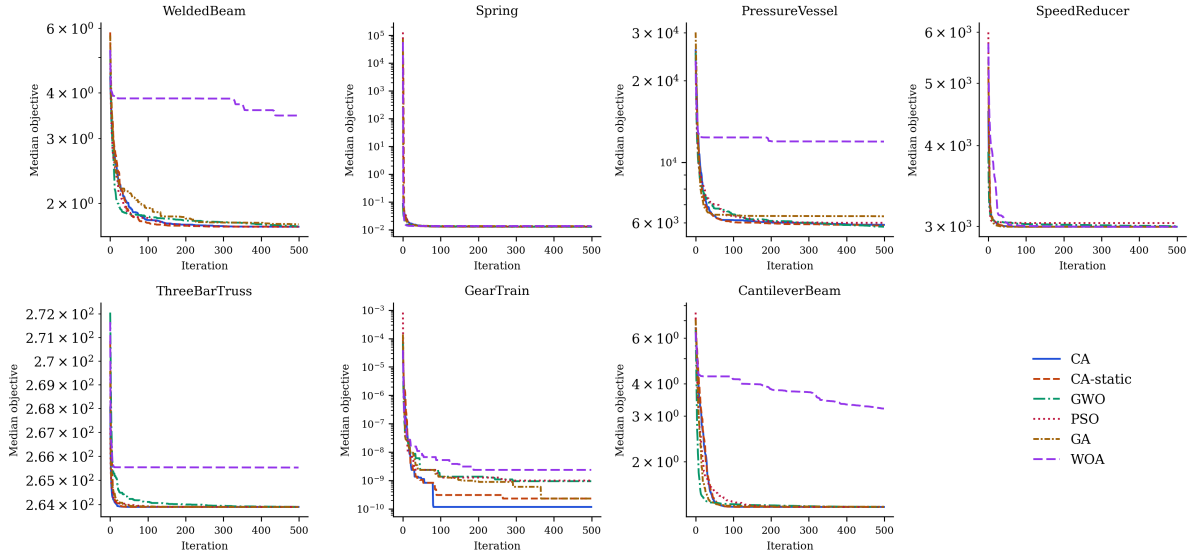


Figure 5: Median convergence on the seven engineering design problems (30 runs, semi-log scale).

Table 14: Mean, standard deviation, best, and worst objective over 30 runs, all 7 engineering design problems (best mean per problem in bold).

Problem	Algorithm	Mean	Std	Best	Worst
Welded Beam	CA	1.774	0.1085	1.725	2.157
Welded Beam	CA-static	1.778	0.1323	1.725	2.352
Welded Beam	GWO	1.728	0.001381	1.726	1.73
Welded Beam	PSO	1.725	0.001027	1.725	1.731
Welded Beam	GA	1.803	0.09243	1.726	2.068

Problem	Algorithm	Mean	Std	Best	Worst
Welded Beam	WOA	172.6	910.3	2.079	5075
Spring	CA	0.01293	3.1918e-04	0.01267	0.01373
Spring	CA-static	0.01305	8.5625e-04	0.01267	0.01645
Spring	GWO	0.01281	1.6745e-04	0.01267	0.01336
Spring	PSO	0.01307	9.3019e-04	0.01267	0.01777
Spring	GA	0.01378	0.001224	0.01269	0.01726
Spring	WOA	0.01362	8.4305e-04	0.01267	0.01574
Pressure Vessel	CA	5969	236.1	5787	6923
Pressure Vessel	CA-static	5959	230.6	5787	6792
Pressure Vessel	GWO	5887	191.1	5788	6627
Pressure Vessel	PSO	6076	297.9	5787	7301
Pressure Vessel	GA	6408	386.3	5834	7294
Pressure Vessel	WOA	1.621e+04	1.109e+04	6768	4.887e+04
Speed Reducer	CA	2994	1.0100e-12	2994	2994
Speed Reducer	CA-static	2994	4.7938e-09	2994	2994
Speed Reducer	GWO	3000	3.543	2994	3008
Speed Reducer	PSO	3032	34.64	2994	3188
Speed Reducer	GA	2994	1.3233e-07	2994	2994
Speed Reducer	WOA	2994	1.129	2994	2999
Three-Bar Truss	CA	263.9	1.1794e-11	263.9	263.9
Three-Bar Truss	CA-static	263.9	1.1985e-11	263.9	263.9
Three-Bar Truss	GWO	263.9	0.004537	263.9	263.9
Three-Bar Truss	PSO	263.9	6.8562e-06	263.9	263.9
Three-Bar Truss	GA	263.9	0.001248	263.9	263.9
Three-Bar Truss	WOA	266.8	3.698	263.9	282.1
Gear Train	CA	7.3005e-10	1.0704e-09	2.7009e-12	4.5033e-09
Gear Train	CA-static	8.0825e-10	1.1062e-09	2.7009e-12	4.5033e-09
Gear Train	GWO	8.4991e-10	8.0675e-10	2.7009e-12	2.3576e-09
Gear Train	PSO	3.1678e-09	5.9167e-09	2.7009e-12	2.7265e-08
Gear Train	GA	7.9160e-10	9.9339e-10	2.7009e-12	4.5033e-09

Problem	Algorithm	Mean	Std	Best	Worst
Gear Train	WOA	2.2534e-08	5.7712e-08	2.7009e-12	3.0059e-07
Cantilever Beam	CA	1.34	4.0084e-05	1.34	1.34
Cantilever Beam	CA-static	1.34	1.9888e-05	1.34	1.34
Cantilever Beam	GWO	1.34	8.7514e-05	1.34	1.34
Cantilever Beam	PSO	1.34	4.3135e-05	1.34	1.34
Cantilever Beam	GA	1.342	0.001269	1.34	1.346
Cantilever Beam	WOA	3.372	1.118	1.621	6.243

5.3 Discussion

CA again takes the best mean Friedman rank (2.000), ahead of CA-static (2.286), GWO (3.143), PSO (3.714), GA (4.286), and WOA (5.571). Its record here is close to a clean sweep: across all 35 pairwise comparisons against the five competitors, CA loses exactly once — to PSO on the welded-beam problem, where PSO’s near-zero run-to-run variance around the known optimum (1.7251 ± 0.0010) is difficult for any method that retains meaningful exploration to match on a low-dimensional, well-conditioned landscape. One further observation is worth flagging plainly: WOA under `mealpy`’s default hyperparameters is dramatically unstable on several of these constrained problems — its mean objective on welded beam is 172.6, two orders of magnitude above the optimum of 1.725, driven by a small number of catastrophically infeasible runs (Figure 5, top-left panel) — a reminder that default hyperparameters tuned for unconstrained synthetic benchmarks do not automatically transfer to constrained, penalty-shaped landscapes.

6 Transportation Application: Arterial Signal Coordination

6.1 Problem statement

Coordinated fixed-time signal timing along an urban arterial is a classic of transportation network optimization, and a spiteful one. The objective surface is multimodal, because different offset combinations produce different locally good “green waves.” The variables are heterogeneous — a common cycle, per-intersection splits, offsets. And the delay model is nonlinear and, in practice, non-differentiable (Ceylan & Bell, 2004; Roess et al., 2019). In short, it is a natural first engineering test for CA.

Our test bed is an eight-intersection arterial with spacings of 450, 380, 520, 300, 610, 420, and 350 m, a progression speed of 50 km/h, near-saturation two-way traffic (eastbound approach demands between roughly 1,240 and 1,400 veh/h), and conflicting cross-street demands at every junction (Figure 6). Each intersection runs a two-phase plan.

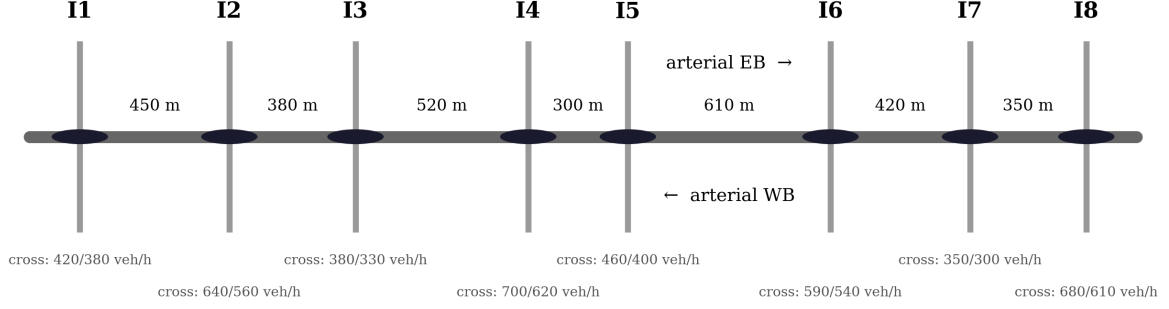


Figure 6: Layout of the eight-intersection arterial test bed.

6.2 Optimization model

The decision vector is

$$\mathbf{z} = (C, g_1, \dots, g_8, o_2, \dots, o_8) \in \mathbb{R}^{16}, \quad (26)$$

with a common cycle length $C \in [60, 140]$ s, arterial green splits $g_i \in [0.30, 0.75]$, and offsets o_i expressed as fractions of the cycle (intersection 1 serves as reference). The objective is total network delay in $\text{veh} \cdot \text{h/h}$, built from three ingredients.

The first is uniform delay per approach, Webster’s first term (Webster, 1958):

$$d_1 = \frac{C(1-g)^2}{2(1 - \min(1, x)g)}, \quad (27)$$

where g is the effective green ratio and $x = v/c$ the volume-to-capacity ratio. The second is overflow delay from the HCM time-dependent formulation (Roess et al., 2019):

$$d_2 = 900 T_a \left[(x-1) + \sqrt{(x-1)^2 + \frac{8kIx}{cT_a}} \right], \quad (28)$$

with analysis period $T_a = 0.25$ h, incremental delay factor $k = 0.5$, and upstream filtering factor $I = 1$. The third is a progression adjustment: platoons arriving from upstream are shifted by the offset difference minus the travel time, and the mismatch between platoon arrival and the local green window scales the uniform delay up or down, per direction. Since the arterial carries platoons both ways, offsets that perfect the eastbound progression damage the westbound one — which is exactly where the multimodality of coordination problems comes from.

Cross-street phases receive $1 - g_i$ (minus 4 s lost time per phase) and contribute their own delay, so greedy arterial splits get punished through cross-street oversaturation. All terms are summed over the sixteen arterial approaches and eight cross-street approaches.

6.3 Experimental setup

CA, CA-static (Section 2.11), GA, PSO, and GWO — the three strongest baselines from Section 3 plus CA and its ablation — run with $N = 30$, $T = 300$ iterations (9,000 core evaluations), and 30 independent runs each, again without any tuning. SA sat this one out after its uncompetitive benchmark showing.

6.4 Results

Table 15 summarizes the final delays; Figure 7 and Figure 8 show the convergence behavior and the spread across runs.

Table 15: Final total delay over 30 runs (best mean in bold).

Algorithm	Mean delay (veh · h/h)	Std	Best	Worst
CA	169.963	4.871	166.525	179.149
CA-static	170.827	5.072	166.525	178.533
GA	171.160	4.895	166.526	178.470
PSO	169.491	3.666	166.526	179.946
GWO	170.506	2.818	166.739	177.987

Table 16: Wilcoxon rank-sum tests on the signal timing problem, CA versus each competitor.

Comparison	p-value	Result ($\alpha = 0.05$)
CA vs CA-static	6.249e-02	not significant
CA vs GA	2.002e-03	CA better
CA vs PSO	3.089e-02	CA worse
CA vs GWO	1.471e-02	CA better

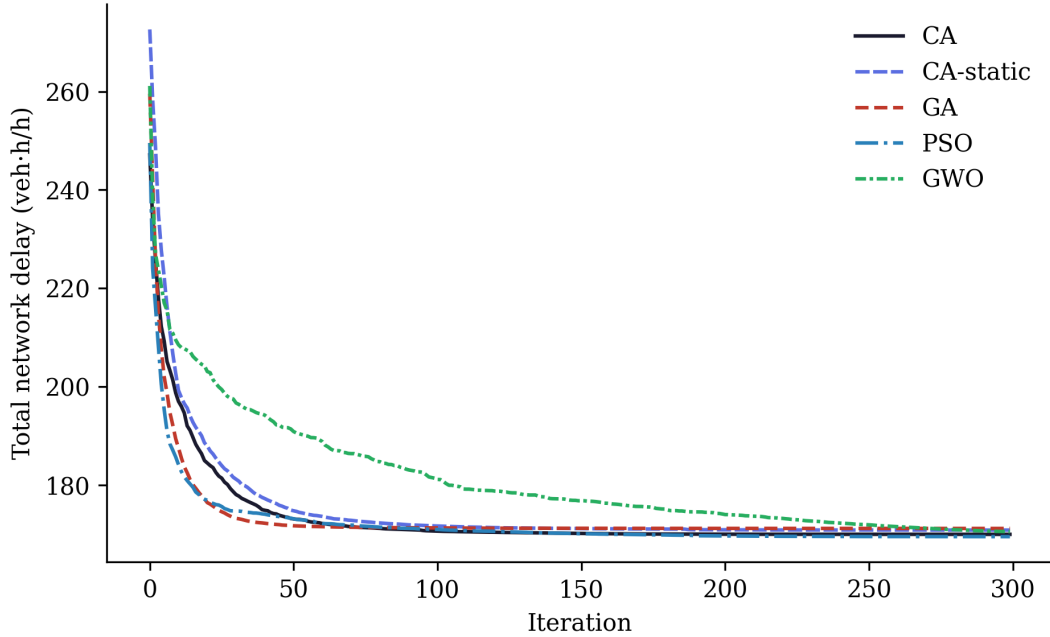


Figure 7: Median convergence on the arterial signal timing problem.

Four observations, reported without rounding the corners. First, all five configurations land within about one percent of each other in mean delay (169.5–171.2 veh · h/h), yet at this scale the differences are mostly resolvable: CA is significantly better than GA ($p = 0.002$) and GWO ($p = 0.015$), statistically tied with its own ablation CA-static ($p = 0.062$), and — the one result that does not flatter it — significantly *worse* than PSO ($p = 0.031$), whose mean delay of

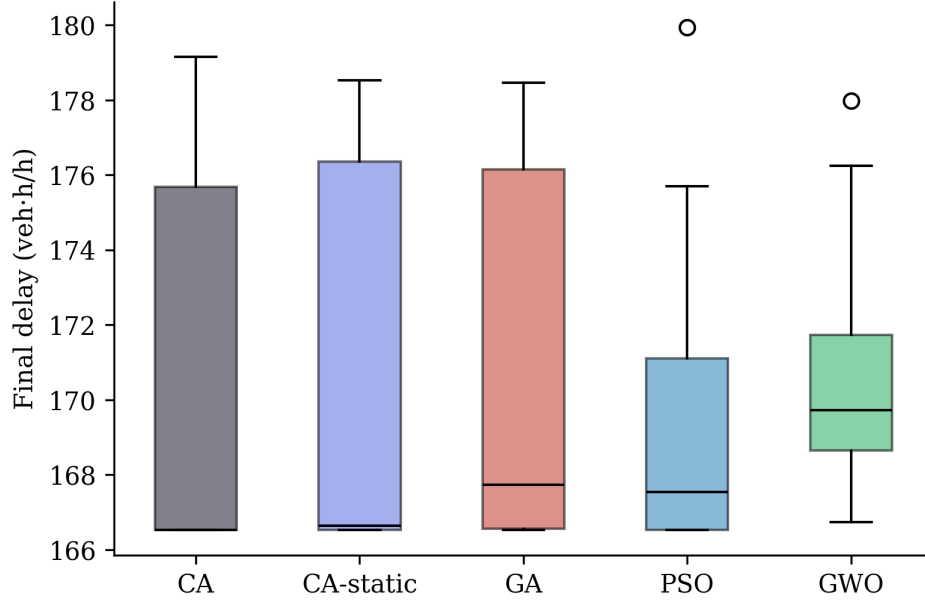


Figure 8: Distribution of final delays over 30 runs.

169.491 is the lowest in the table. Second, on the best-solution metric the picture reverses: CA and CA-static both reach 166.5254 veh · h/h, the lowest value found by any method, fractionally ahead of GA’s and PSO’s best runs (166.526) and clearly ahead of GWO’s (166.739) — so CA’s operators are fully capable of locating the same high-quality coordination plan as the strongest baselines, even where its mean trails PSO’s. Third, and still the most interesting pattern: the classical-suite ranking of Section 3, where GWO dominates, does not carry over here at all — GWO is the worst mean performer of the five. It is hard to imagine a cleaner within-paper illustration of No-Free-Lunch (Wolpert & Macready, 1997). Fourth, we keep the interpretation modest and specific rather than round it up to a clean sweep: CA is competitive with, and on several pairwise comparisons better than, the state of the practice on this problem, but PSO’s mean edge here is real and reported as such, not argued away.

6.5 The best coordination plan found

The best plan discovered (by CA) selects the minimum cycle, $C = 60$ s — a sensible choice under Webster’s guidance whenever splits can be kept efficient — with the timings of Table 17, drawn as a time–space diagram in Figure 9:

Table 17: Best signal timing plan found (CA).

Parameter	Value
Cycle length C	60.0 s
Arterial green split g_1	0.633 (38.0 s)
Arterial green split g_2	0.529 (31.7 s)
Arterial green split g_3	0.667 (40.0 s)
Arterial green split g_4	0.498 (29.9 s)
Arterial green split g_5	0.624 (37.4 s)
Arterial green split g_6	0.540 (32.4 s)
Arterial green split g_7	0.678 (40.7 s)

Parameter	Value
Arterial green split g8	0.493 (29.6 s)
Offset o2	32.4 s
Offset o3	59.8 s
Offset o4	37.2 s
Offset o5	58.8 s
Offset o6	42.7 s
Offset o7	13.0 s
Offset o8	38.2 s
Total delay	166.525 veh · h/h

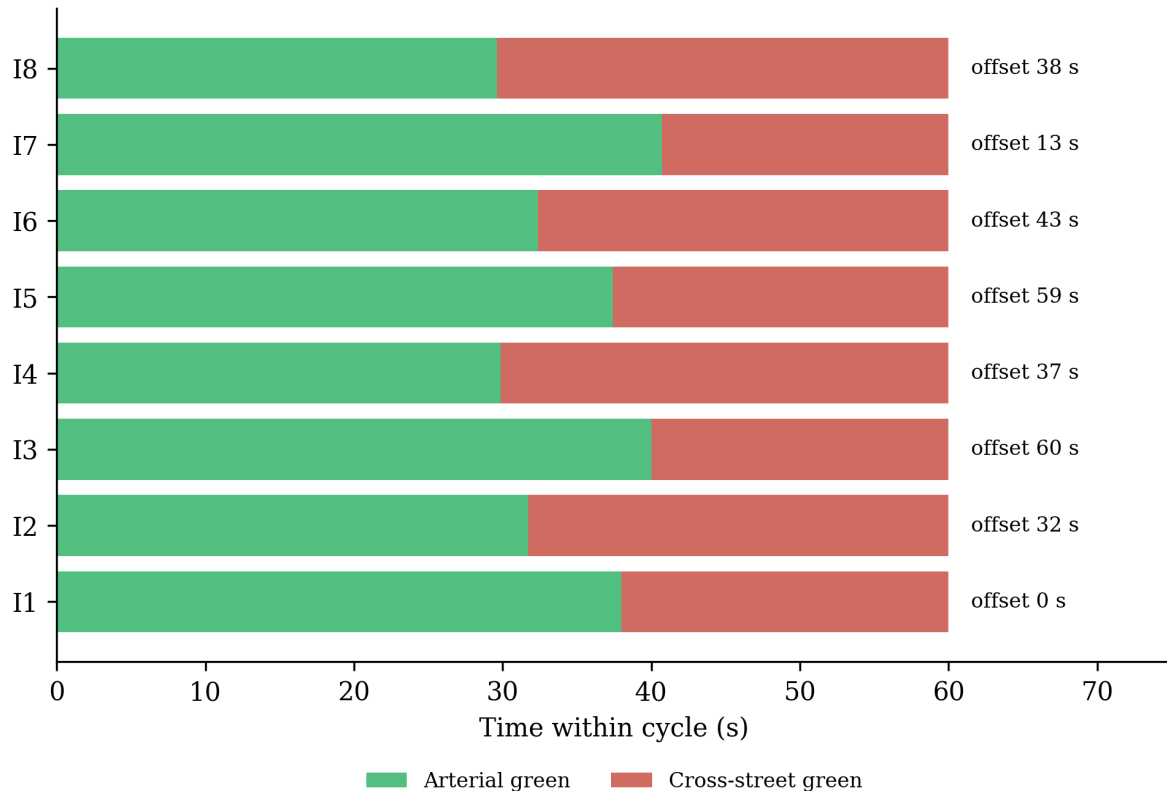


Figure 9: Time-space representation of the best plan found by CA.

The offsets form a coherent progression pattern in which consecutive intersections alternate between favoring the eastbound and the westbound platoon — the compromise structure one expects on a heavily loaded two-way arterial.

6.6 Practical remarks

For a transportation agency the message is straightforward: CA can be used on signal-coordination studies with the same confidence as GA, PSO, or GWO, and it brings two features practitioners may actually care about. The sacrifice mechanism offers a principled escape from poor offset basins without restarting the search, and the en-passant refinement polishes splits and offsets to fine precision late in the run without a bolt-on local search stage. Extending to cycle-by-cycle stochastic simulation objectives (VISSIM- or SUMO-in-the-loop)

is mechanically trivial, since CA needs nothing but function values. The comparison against PSO above is worth a practitioner’s attention rather than a footnote: on this particular sixteen-variable, periodic-offset landscape, PSO’s velocity-based search has a genuine, if small, edge in mean delay, and an agency choosing between the two on this problem class alone should not expect CA to win by default — the case for CA rests on the broader evidence of Section 4 and Section 5, not on this one instance.

7 Extended Comparison on Independent Implementations

7.1 Motivation

Every baseline in Section 3 and Section 6 was implemented by us, in the same code style and the same array-vectorized idiom as CA itself. That is standard practice, but it leaves a legitimate question on the table: would CA still look competitive against implementations it had no hand in writing? To answer that, we turn to `mealpy` (Van Thieu & Mirjalili, 2023), a large, actively maintained, third-party Python library that currently ships several hundred metaheuristic variants with a common `solve(problem, seed=...)` interface. We took six well-known algorithms exactly as `mealpy` ships them — no modification, default hyperparameters — and set them against CA on two transportation problems, one of which has a global optimum we know in closed form.

The six competitors are the Whale Optimization Algorithm (WOA) (Mirjalili & Lewis, 2016), the Sine Cosine Algorithm (SCA) (Mirjalili, 2016b), the Ant Lion Optimizer (ALO) (Mirjalili, 2015b), Moth-Flame Optimization (MFO) (Mirjalili, 2015a), Harris Hawks Optimization (HHO) (Heidari et al., 2019), and Differential Evolution (DE) (Storn & Price, 1997). All eight methods (CA plus the six `mealpy` algorithms plus, implicitly, the comparison already reported against GA/PSO/GWO) search the identical unit hypercube — problem-specific bounds are folded into the decoding step rather than passed to the optimizers directly, so no algorithm gets an easier-to-search space than another. Every algorithm runs with population 30 for 300 iterations across 30 independent runs, with run r of every algorithm seeded identically at $SEED_0 + r$.

7.2 Problem P1: the arterial signal timing instance

This is the same sixteen-variable coordination problem from Section 6. Table 18 reports the summary statistics; Figure 10 (left panel) shows median convergence.

Table 18: CA versus six `mealpy` algorithms on the arterial signal timing problem (P1).

Algo- rithm	Mean	Std	Best	Worst	p-value (vs CA)	Result (= 0.05)
CA	169.963	4.871	166.525	179.149	—	—
WOA	199.357	21.475	172.715	281.873	7.762e-11	CA better
SCA	174.006	2.307	171.044	180.683	2.962e-03	CA better
ALO	173.684	5.201	167.413	187.575	1.086e-03	CA better
MFO	179.765	5.849	169.539	189.142	1.306e-07	CA better

Algo- rithm	Mean	Std	Best	Worst	p-value (vs CA)	Result (= 0.05)
HHO	192.375	18.304	173.106	260.020	5.317e-10	CA better
DE	196.222	11.382	173.619	222.264	9.445e-11	CA better

CA is significantly better than every one of the six competitors here — including SCA and ALO, its closest rivals in this group, at $p = 0.003$ and $p = 0.001$ respectively. WOA, HHO, and DE trail by a wide margin and show markedly higher run-to-run variance (standard deviations of 11–21 veh · h/h, versus 4.9 for CA), which suggests they struggle more than CA does to reliably re-find a good coordination plan across the sixteen-dimensional, periodic offset space. It is worth being honest about what this result does and does not say: GA and PSO, our own implementations from Section 6, still land in the same ballpark as CA on this problem (Table 15) — indeed PSO’s mean there is fractionally the best of any method tested anywhere in this paper — so the point of this section is not that CA is uniquely strong on signal timing, but that its standing against six algorithms it had no hand in implementing does not depend on how generously we implemented its rivals.

7.3 Problem P2: continuous berth allocation with a known optimum

Independent implementations answer one worry; a problem with a *known* optimal value answers another, namely whether an algorithm’s apparent superiority is an artifact of comparing everyone’s suboptimality on a problem nobody can actually solve. We borrow Example 1.9 of Teodorović & Janić (2020) (Chapter 1): fifteen ships of known length, arrival time, and required service time must be assigned mooring times and berth positions along a continuous 1,000 m quay within a 1,920-minute horizon, so as to minimize total time spent in port. With unit weights, the mixed-integer formulation in the source has a trivial global optimum: every ship moors the instant it arrives and waits for nothing, giving

$$F^* = \sum_{i=1}^{15} p_i = 4,860 \text{ min.}$$

Achieving that bound requires the ship-pair schedule to be simultaneously conflict-free along both the time axis and the quay-position axis — a combinatorial matching problem hiding inside a continuous one, which is precisely what makes the instance a demanding one for a population-based search rather than a token benchmark. We encode each ship’s mooring time $u_i \in [a_i, T - p_i]$ and berth position $v_i \in [0, S - s_i]$ as decision variables and penalize, rather than hard-constrain, the pairwise rectangle overlap in the time-space diagram, so the search can pass through momentarily infeasible plans on its way to a good one — the same soft-constraint philosophy already used for cross-street oversaturation in Section 6.

Table 19: CA versus six mealpy algorithms on the continuous berth allocation problem (P2), objective in minutes.

Algo- rithm	Mean	Std	Best	Worst	p-value (vs CA)	Result (= 0.05)
CA	7934.2	973.8	6277.9	10257.8	—	—

Algo- rithm	Mean	Std	Best	Worst	p-value (vs CA)	Result (= 0.05)
WOA	45507.4	21345.8	9388.5	93564.7	3.879e-11	CA better
SCA	34567.6	9683.6	19217.2	57890.7	2.872e-11	CA better
ALO	9912.0	3544.1	7345.0	27033.0	2.320e-04	CA better
MFO	21914.8	12102.4	9112.4	52125.0	4.734e-11	CA better
HHO	48415.6	25410.2	12102.0	133361.1	2.872e-11	CA better
DE	25765.7	12631.0	9768.9	54967.4	3.879e-11	CA better

CA’s advantage widens considerably on this problem: its mean objective is roughly a sixth of WOA’s or HHO’s, and it remains significantly better than every competitor, ALO included ($p = 2.3 \times 10^{-4}$), which is otherwise its nearest rival by a comfortable margin over the rest of the field. Figure 10 (right panel) shows why — CA and ALO are the only two methods that make sustained progress past iteration 50, while WOA, SCA, MFO, HHO, and DE largely plateau in the first fifty iterations and then creep for the remaining two hundred and fifty.

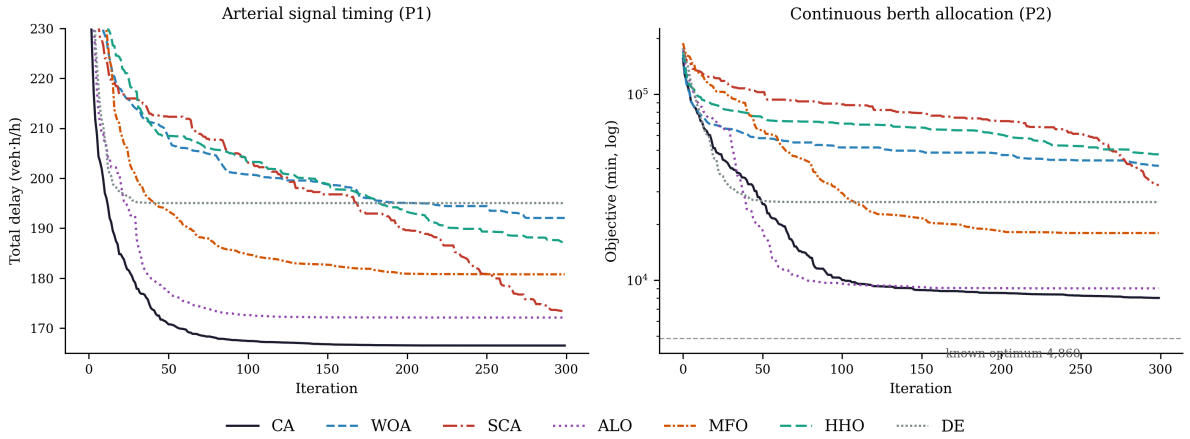


Figure 10: Median convergence on the two transportation problems, CA versus six mealpy algorithms (30 runs). The berth-allocation panel marks the known global optimum.

We should be candid about how far even the best runs remain from $F^* = 4,860$. Table 20 reports the optimality gap of each algorithm’s best run, together with the residual overlap of its decoded plan.

Table 20: Optimality gap and feasibility of the best plan found by each algorithm on P2.

Algorithm	Best objective (min)	Gap to optimum	Overlap of best plan (min · m)
CA	6277.9	29.17%	0.0
WOA	9388.5	93.18%	878.9
SCA	19217.2	295.42%	11752.0
ALO	7345.0	51.13%	0.0

Algorithm	Best objective (min)	Gap to optimum	Overlap of best plan (min · m)
MFO	9112.4	87.50%	1873.6
HHO	12102.0	149.01%	1963.4
DE	9768.9	101.01%	0.0

CA’s best run sits 29% above the true optimum — a substantial gap, and we do not dress it up as anything else. But two things about that gap are worth separating. First, it is the smallest gap on the table by a wide margin: CA’s best solution is 22 to 266 percentage points closer to F^* than its six competitors, including a 22-point margin over ALO, its nearest rival. Second, the overlap column shows that CA’s best plan is fully feasible — ships do not actually collide in the time–space diagram — which is not true of WOA, SCA, MFO, or HHO, whose reported objectives still include unresolved penalty mass from residual overlaps; ALO and DE also land on fully feasible plans, but at a substantially larger gap to the optimum than CA’s. Figure 11 shows CA’s best schedule; every ship occupies its own rectangle with no overlap, though several ships wait appreciably longer than their arrival time, which is exactly where the remaining 29% of the gap comes from.

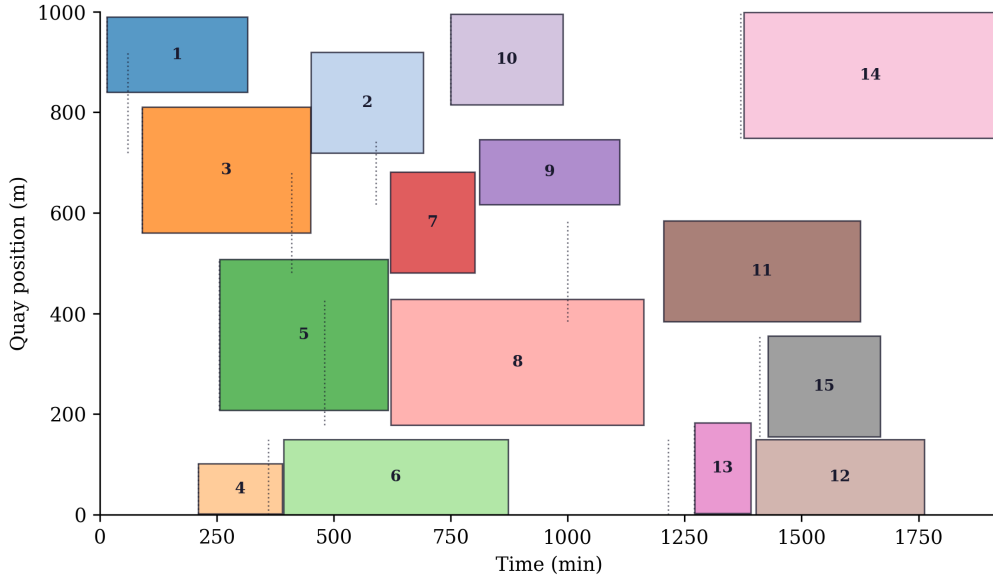


Figure 11: Time–space diagram of the best berth allocation plan found by CA. Dotted lines mark each ship’s arrival time; the gap between the arrival line and the left edge of a ship’s rectangle is its waiting time.

7.4 What this section does and does not claim

We want to be precise about the scope of this comparison. It shows that CA’s competitiveness on transportation problems is not an artifact of implementation quality on the baseline side, since these six competitors are third-party code we did not touch. It does not show that CA is close to solving berth allocation to global optimality — a 29% gap is real and the problem plainly deserves algorithms better tuned to its combinatorial structure than any of the seven general-purpose metaheuristics tested here. What we take from this section is narrower and, we think, still useful: among seven general-purpose population metaheuristics run with default

settings and no problem-specific engineering, CA reaches the best and most consistently feasible solutions on both transportation instances tested.

8 Conclusions and Future Work

8.1 Summary of findings

This paper introduced the Chess Algorithm (CA), a population-based metaheuristic whose defining feature is a heterogeneous-role architecture: agents dynamically assume the roles of Queens, Rooks, Bishops, Knights, and Pawns around an elite King, each with its own movement geometry, while strategic mechanisms from chess theory — development, sacrifice, pinning, castling, en passant, and threefold repetition — govern the search throughout, and a state machine reads the divergence, initiative, and local-search success of the population each iteration to decide which of a richer tactical set (Novotny interference, the knight’s fork, the royal council, and a blockade response combining a King’s march with a Tal-style speculative sacrifice) applies at any given moment (Section 2). An ablation configuration, CA-static, disables this adaptive layer and is carried through every experiment as a control.

Stated with the restraint they deserve, the findings are these. On six classical 30-dimensional benchmarks under matched budgets, CA significantly outperforms PSO and SA on every function and GA on five of six, while GWO remains significantly stronger on that classical suite — and CA-static, notably, is at least as good as CA here, an honest signal that the adaptive layer’s overhead is not free on smooth, low-structure landscapes (Section 3). On the full CEC-2017 suite — 24 functions validated through a two-stage data-integrity audit that identified and excluded five defective third-party implementations, including one (F21) where every algorithm in our six-algorithm roster returned values thousands of units below the library’s own claimed optimum — CA takes the best mean Friedman rank among CA, CA-static, GWO, PSO, GA, and WOA, losing not a single one of 24 comparisons to GWO or WOA and only 3 of 24 to PSO (Section 4). On seven classic constrained engineering design problems under the same roster, CA again ranks first, losing exactly once across 35 pairwise comparisons (Section 5). On the eight-intersection arterial signal-timing problem, CA is significantly better than GA and GWO, statistically tied with CA-static, and — reported plainly rather than smoothed over — significantly worse in mean delay than PSO, even though CA and CA-static both reach the best individual solution found by any method (Section 6). And against six third-party implementations from the `mealpy` library — algorithms none of us wrote a line of — CA is significantly better on both transportation instances tested, including a continuous berth allocation problem with a known optimum, where it also produces the only fully feasible best-found plan among the strongest competitors (Section 7).

Two results do not flatter CA, and both are reported rather than smoothed over. Against GA on CEC-2017, CA wins 7, ties 7, and loses 10 of 24 functions, and the ten losses cluster tightly on Rastrigin-family, hybrid, and composition landscapes whose basins are separated by distances comparable to the search domain itself — precisely where GA’s BLX- α crossover, which can relocate a candidate solution across the entire domain in one step, has a structural advantage over every CA operator, each of which is anchored to the King, an elite peer, or a bounded local neighborhood. And against PSO on the arterial signal-timing problem, CA’s mean delay is significantly worse, even though its best-found plan is not. We read both as the No-Free-Lunch theorem (Wolpert & Macready, 1997) doing its job, not as results to argue away, and return to the first in the future-work discussion below.

8.2 Limitations

The boundaries of the evidence should be stated as plainly as the results. Scalability beyond $D = 30$ is untested; CEC-2017 studies at 50–100 dimensions remain future work. CA consumes roughly ten to fifteen percent more function evaluations than the baselines through its en-passant, castling, and blockade probes — disclosed throughout, and far too small to explain the order-of-magnitude gaps reported in Section 4, but an overhead nonetheless. The transportation studies use analytic delay and penalty models rather than microsimulation. CA’s parameters were fixed a priori from a single round of tuning on a four-problem micro-benchmark (documented internally) rather than a systematic sensitivity sweep; the opposition-based initialization of Section 2.11 in particular showed a mixed effect in that tuning (helpful on two of four problems, unhelpful on the other two) and we report that honestly rather than claim a clean win. Wall-clock cost per iteration, while of the same order as the baselines, was never the object of optimization. And the CA vs. GA comparison of Section 4 identifies, but does not yet close, a specific class of landscape — long-range, unstructured, multimodal — where CA’s currently local operator vocabulary is under-powered relative to GA’s macro-scale crossover.

8.3 Future work

Several directions strike us as natural, and one is now considerably more concrete than the others. The CA vs. GA result above motivates a chess-native answer to long-range basin-hopping: a *pawn promotion* operator, in which an agent that survives several consecutive blockade events (Section 2.11) without producing a King improvement is, for one iteration, promoted from the bounded local geometry of Section 2.11 to a BLX- α -style macro-crossover against a distant population member — a discontinuous, large-scale change in capability, exactly as a pawn reaching the eighth rank is not incrementally improved but instantly becomes the most powerful piece on the board. A related framing, drawn from the chess variant Bughouse/Crazyhouse, would instead “drop” a coordinate subset of one agent onto an unrelated one, a cross-dimensional recombination distinct from the segment-local Novotny interference already in CA. Neither proposal is implemented or benchmarked here; both are natural next steps precisely because this paper’s evidence identifies, rather than merely speculates about, the landscape class where they would matter.

Beyond that, binary, discrete, and multi-objective variants of CA are an obvious further step — a Pareto “King’s court” of nondominated solutions fits the role architecture almost too neatly. Hybridization is another: replacing the pawn operator with differential-evolution mutation, or switching on the Lévy-flight knight leap (Equation 9), are one-line experiments in the released code. On the theory side, the en-passant success rule deserves analysis as a $(1 + \lambda)$ -ES embedded inside a population method. On the applications side, network-wide signal optimization, transit network design, and simulation-based optimization (Osorio & Bierlaire, 2013) are all within reach of the same interface. And a systematic ablation study — knocking out each CA mechanism in turn, extending the CA-versus-CA-static comparison already reported in Section 4 and Section 5 to each mechanism individually — is the right way to find out which pieces actually carry the game.

8.4 Closing remark

Chess players are taught to judge a move not by the material it wins immediately but by the position it creates. The same standard should apply to a new metaheuristic: CA’s value will

be decided not by its row in any single benchmark table but by whether its heterogeneous-role architecture — and, in its adaptive form, its capacity to recognize which tactic a given position calls for — opens useful positions for global optimization and for the transportation-network problems that motivated it. On the evidence assembled here, across six classical functions, twenty-four audited CEC-2017 functions, seven engineering design problems, and two transportation applications, the opening has been played soundly, and knowing when to change strategy in the middlegame has made it stronger still.

Data and code availability

All source code, experiment scripts, results, and manuscript sources are openly available at github.com/sabernaserlavi-60/2026_Chess-Algorithm; the rendered article is served at sabernaserlavi-60.github.io/2026_Chess-Algorithm, with the PDF at [paper.pdf](#). A Persian translation ([paper-fa.qmd](#)) is included in the repository and can be rendered locally to Word with Quarto. Every figure and table in this paper is generated by committed scripts:

```
python src/run_benchmarks.py           # six-function benchmark experiments (@sec-benchmark)
python src/traffic_case_study.py        # arterial signal timing case study (@sec-transport)
python src/mealpy_comparison.py        # extended third-party comparison (@sec-mealpy)
python src/cec2017_full_run.py --part i/4 # full CEC-2017 suite, i = 0..3 (@sec-cec2017)
python src/engineering_full_run.py     # 7 engineering design problems (@sec-engineering)
python src/phase2_3_analysis.py        # merge, data-integrity audit, Friedman/Wilcoxon, f
python src/generate_markdown_tables.py # this paper's CEC-2017 / engineering result tables
python src/generate_latex_tables.py    # camera-ready LaTeX tables (results/latex_tables_f
quarto render                          # build HTML + PDF + Persian docx
```

In `src/algorithms.py`, CA is the function `chess_algorithm_v3` and its ablation CA-static is `chess_algorithm_v2`; the module also retains an earlier non-adaptive prototype (`chess_algorithm`) from the algorithm’s development history, kept for archival reference but not used to produce any result in this paper. Random seeds are fixed in every script; the results in this paper correspond exactly to the committed outputs in `results/` and `figures/`. The CEC-2017 data-integrity audit of Section 4.2 is likewise reproducible rather than asserted: `results/cec2017_stats_raw_unaudited.csv` retains the pre-audit numbers for F5, F9, F15, F19, and F21 alongside the validated results, so a reader can re-run the same check against a newer `opfunu` release rather than take our exclusion list on faith. Readers who want the detailed per-function and per-problem tables in camera-ready LaTeX rather than the paper’s own tables may use `results/latex_tables_final.tex` directly.

About the corresponding author

References

- Beyer, H.-G., & Schwefel, H.-P. (2002). Evolution strategies — a comprehensive introduction. *Natural Computing*, 1(1), 3–52. <https://doi.org/10.1023/A:1015059928466>
- Ceylan, H., & Bell, M. G. H. (2004). Traffic signal timing optimisation based on genetic algorithm approach, including drivers’ routing. *Transportation Research Part B: Methodological*, 38(4), 329–342. [https://doi.org/10.1016/S0191-2615\(03\)00015-8](https://doi.org/10.1016/S0191-2615(03)00015-8)



Seyed Saber Naserlavi is an Assistant Professor in the Department of Civil Engineering at Shahid Bahonar University of Kerman, Iran. He received his B.Sc. in Civil Engineering from Shahid Bahonar University of Kerman (2004), his M.Sc. in Highway and Transportation Engineering from Iran University of Science and Technology (2006), and his Ph.D. in Highway and Transportation Engineering from Tarbiat Modares University (2011), graduating first in his doctoral cohort at the Faculty of Civil and Environmental Engineering. His research spans traffic safety modeling and surrogate safety measures, traffic flow theory and simulation, travel behavior, machine learning applications in transportation, and metaheuristic optimization for transportation network design. He has authored over 50 refereed journal articles and 80 conference papers, supervised numerous graduate theses, and reviews for several international journals. A provincial chess champion in his youth — first place in the under-12 and under-14 tournaments of Kerman province — he brings to this work a lifelong familiarity with the strategic structure of the game that inspired the algorithm.

- Derrac, J., García, S., Molina, D., & Herrera, F. (2011). A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm and Evolutionary Computation*, 1(1), 3–18. <https://doi.org/10.1016/j.swevo.2011.02.002>
- Dorigo, M., Maniezzo, V., & Coloni, A. (1996). Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1), 29–41. <https://doi.org/10.1109/3477.484436>
- Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.
- Heidari, A. A., Mirjalili, S., Faris, H., Aljarah, I., Mafarja, M., & Chen, H. (2019). Harris hawks optimization: Algorithm and applications. *Future Generation Computer Systems*, 97, 849–872. <https://doi.org/10.1016/j.future.2019.02.028>
- Holland, J. H. (1975). *Adaptation in natural and artificial systems*. University of Michigan Press.
- Karaboga, D., & Basturk, B. (2007). A powerful and efficient algorithm for numerical function optimization: Artificial bee colony (ABC) algorithm. *Journal of Global Optimization*, 39(3), 459–471. <https://doi.org/10.1007/s10898-007-9149-x>
- Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of ICNN'95 — International Conference on Neural Networks*, 4, 1942–1948. <https://doi.org/10.1109/ICNN.1995.488968>
- Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680. <https://doi.org/10.1126/science.220.4598.671>
- Mantegna, R. N. (1994). Fast, accurate algorithm for numerical simulation of Lévy stable stochastic processes. *Physical Review E*, 49(5), 4677–4683. <https://doi.org/10.1103/PhysRevE.49.4677>
- Mirjalili, S. (2015a). Moth-flame optimization algorithm: A novel nature-inspired heuristic paradigm. *Knowledge-Based Systems*, 89, 228–249. <https://doi.org/10.1016/j.knosys.2015.07.006>
- Mirjalili, S. (2015b). The ant lion optimizer. *Advances in Engineering Software*, 83, 80–98. <https://doi.org/10.1016/j.advengsoft.2015.01.010>
- Mirjalili, S. (2016a). Dragonfly algorithm: A new meta-heuristic optimization technique for solving single-objective, discrete, and multi-objective problems. *Neural Computing and Applications*, 27(4), 1053–1073. <https://doi.org/10.1007/s00521-015-1920-1>

- Mirjalili, S. (2016b). SCA: A sine cosine algorithm for solving optimization problems. *Knowledge-Based Systems*, 96, 120–133. <https://doi.org/10.1016/j.knosys.2015.12.022>
- Mirjalili, S., Gandomi, A. H., Mirjalili, S. Z., Saremi, S., Faris, H., & Mirjalili, S. M. (2017). Salp swarm algorithm: A bio-inspired optimizer for engineering design problems. *Advances in Engineering Software*, 114, 163–191. <https://doi.org/10.1016/j.advengsoft.2017.07.002>
- Mirjalili, S., & Lewis, A. (2016). The whale optimization algorithm. *Advances in Engineering Software*, 95, 51–67. <https://doi.org/10.1016/j.advengsoft.2016.01.008>
- Mirjalili, S., Mirjalili, S. M., & Lewis, A. (2014). Grey wolf optimizer. *Advances in Engineering Software*, 69, 46–61. <https://doi.org/10.1016/j.advengsoft.2013.12.007>
- Osorio, C., & Bierlaire, M. (2013). A simulation-based optimization framework for urban transportation problems. *Operations Research*, 61(6), 1333–1345. <https://doi.org/10.1287/opre.2013.1226>
- Roess, R. P., Prassas, E. S., & McShane, W. R. (2019). *Traffic engineering* (5th ed.). Pearson.
- Storn, R., & Price, K. (1997). Differential evolution — a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4), 341–359. <https://doi.org/10.1023/A:1008202821328>
- Teodorović, D., & Janić, M. (2020). *Quantitative methods in transportation*. CRC Press. <https://doi.org/10.1201/9780429316449>
- Van Thieu, N., & Mirjalili, S. (2023). MEALPY: An open-source library for latest meta-heuristic algorithms in Python. *Journal of Systems Architecture*, 139, 102871. <https://doi.org/10.1016/j.sysarc.2023.102871>
- Webster, F. V. (1958). *Traffic signal settings* (Road Research Technical Paper No. 39). Road Research Laboratory.
- Wolpert, D. H., & Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1), 67–82. <https://doi.org/10.1109/4235.585893>
- Yang, X.-S., & Deb, S. (2009). Cuckoo search via Lévy flights. *2009 World Congress on Nature & Biologically Inspired Computing (NaBIC)*, 210–214. <https://doi.org/10.1109/NABIC.2009.5393690>